

Gert v2 Design Document

Gert Team

April 18, 2026

Contents

1	Overview	1
1.1	Problem Statement	1
1.2	Why v2: What v1 Got Right, and What Constrained It	1
1.3	Scope	2
1.4	Design Priorities	2
1.5	Document Roadmap	3
1.6	Success Definition	3
2	Goals and Non-Goals	5
2.1	Goals	5
2.2	Non-Goals	7
3	Architecture	8
3.1	Primary Components	8
3.1.1	Parser	8
3.1.2	Planner	10
3.1.3	Runtime Core	11
3.1.4	Extension Host	13
3.1.5	API / Adapter Layer	14
3.2	Data Flow	16
3.2.1	Phase 1: Parse	16
3.2.2	Phase 2: Plan	16
3.2.3	Phase 3: Run Initialization	17
3.2.4	Phase 4: Step Execution Loop	17
3.2.5	Phase 5: Run Completion	19
3.3	Lifecycle Model	19
3.3.1	Run Start	19
3.3.2	Step Advancement	20

3.3.3	Pause and Resume	20
3.3.4	Cancellation	20
3.3.5	Crash Recovery	21
3.4	Concurrency Model	21
3.4.1	Per-Process Run Isolation	21
3.4.2	Goroutine Model	21
3.4.3	Thread Safety at Component Boundaries	22
3.5	gert serve Integration	22
3.5.1	Architectural Position	22
3.5.2	Event Forwarding	23
3.5.3	Crash Recovery in gert serve	23
3.5.4	RPC Method Summary	23
4	Schema vNext	24
4.1	Schema Identity and Versioning	24
4.1.1	apiVersion Values	24
4.1.2	Self-Description via \$schema	25
4.1.3	Versioning Strategy	25
4.1.4	Compatibility Contract: v1 $\rightarrow$ v2	27
4.1.5	gert migrate Behavior	27
4.2	Core Runbook Fields (v2)	28
4.2.1	Top-Level Field Inventory	28
4.2.2	toolRefs	30
4.2.3	Minimal Valid v2 Runbook	30
4.2.4	Complete Annotated Example	31
4.3	Input and Output Declarations	34
4.3.1	Input Declarations	34
4.3.2	Output Declarations	35
4.3.3	Expression Language	35
4.4	Step Types (v2 Inventory)	36
4.4.1	Common Step Fields	37
4.4.2	Step Type: cli	38
4.4.3	Step Type: manual	39
4.4.4	Step Type: tool	40
4.4.5	Step Type: invoke	42
4.4.6	Step Type: branch	42

4.4.7	Step Type: iterate	44
4.4.8	Step Type: parallel	46
4.4.9	Step Type: assert	47
4.4.10	Step Type: compensate	48
4.4.11	Step Type: end	49
4.5	Tool Definition Schema (v2)	49
4.5.1	Tool Definition Field Inventory	49
4.6	Provider Definition Schema (v2)	51
4.7	Namespace Convention for Extensions	52
4.7.1	The Extension Field Problem	52
4.7.2	Convention: <code>x-<namespace></code> : at the Field Level	52
4.7.3	Extension Validation	53
4.7.4	Extension Registration Format	53
4.8	Structural vs. Semantic Validation	54
4.8.1	Phase 1: Structural Validation	54
4.8.2	Phase 2: Semantic Validation	55
4.9	Schema Artifacts	57
4.9.1	Shipped Artifacts	57
4.9.2	<code>gert schema export</code>	57
4.9.3	VS Code Integration	57
4.9.4	Schema Version Manifest	58
5	Extension Runtime	59
5.1	Architecture Overview	59
5.2	Capability Taxonomy	60
5.3	Extension Manifest Format	60
5.4	Extension Discovery	61
5.5	Handshake and Lifecycle Protocol	62
5.5.1	<code>extension/initialize</code> (host → extension)	62
5.5.2	<code>contributions/list</code> (host → extension)	64
5.5.3	<code>extension/ping</code> (host → extension)	64
5.5.4	<code>extension/shutdown</code> (host → extension)	65
5.5.5	Lifecycle State Machine	65
5.6	Versioning and Compatibility	66
5.7	Sandboxing and Process Isolation	66
5.7.1	Process Isolation	66

5.7.2	Capability Enforcement	67
5.7.3	Timeout and Crash Handling	67
6	Tool Runtime	70
6.1	Tool Discovery	70
6.1.1	Name Resolution Order	71
6.2	Tool Definition Schema (v2)	71
6.2.1	Transport-Specific Fields	72
6.3	Transport Layer	73
6.3.1	stdio Transport	73
6.3.2	stdio-jsonrpc Transport	74
6.3.3	MCP Transport	75
6.4	Invocation Context	76
6.5	Capability Gate	76
6.6	Output Capture	76
6.7	MCP Tool Integration	77
6.7.1	MCP Server Declaration	77
6.7.2	MCP Discovery	77
6.7.3	MCP Tool Invocation	78
6.7.4	MCP vs stdio-jsonrpc Differences	78
6.8	Cancellation and Timeout	79
6.9	Tool Versioning	80
7	Runtime Events and Determinism	81
7.1	Event Categories	81
8	Security and Trust	82
8.1	Trust Model	82
8.1.1	Runtime Hardening	82
9	Testing and Acceptance	83
9.1	Test Framework	83
9.1.1	Standard Library and testify	83
9.1.2	Test Packages	84
9.2	The <code>gert test</code> Feature	84
9.2.1	Scenario Replay Testing	84
9.2.2	Test Assertions (<code>test.yaml</code>)	85

9.2.3	v1 Scenario Compatibility	85
9.3	Contract Tests	86
9.3.1	Core Runtime Contract	86
9.3.2	Schema Contract Tests	86
9.3.3	Extension Manifest and Handshake Tests	87
9.3.4	Tool Invocation Envelope Tests	87
9.3.5	Event Bus Determinism Tests	87
9.4	Integration Test Strategy	88
9.4.1	End-to-End Fixture Runs	88
9.4.2	Fixture Runbook Corpus	88
9.4.3	Trace Assertion Helpers	89
9.5	Regression Testing	89
9.5.1	v1 Runbook Corpus as Fixtures	89
9.5.2	Golden Trace Comparisons	90
9.6	Performance Acceptance Criteria	90
9.7	Extension Test Harness	91
9.7.1	Overview	91
9.7.2	Harness Capabilities	91
9.7.3	Mocking the Extension Transport	92
9.8	Acceptance Criteria	92
9.9	CI/CD Integration	93
9.9.1	PR Gate (every pull request)	93
9.9.2	Integration Gate (merge to main)	93
9.9.3	Nightly Gate	93
10	Open Questions	94
11	Migration and Compatibility	100
11.1	TODO	100
12	Governance and Policy	101
12.1	Governance Primitives	101
12.1.1	Command Allowlist (<code>allowed_commands</code>)	101
12.1.2	Command Denylist (<code>denied_commands</code>)	102
12.1.3	Environment Variable Blocking (<code>deny_env_vars</code>)	102
12.1.4	Output Redaction (<code>redact</code>)	103

12.1.5	Approval Gates	103
12.2	Policy Evaluation Model	104
12.3	Approval Gate Design	104
12.3.1	Schema Declaration	104
12.3.2	UX Contract	105
12.3.3	Approval Decision Protocol	105
12.3.4	Approval Recording in the Trace	106
12.3.5	Timeout and Escalation	107
12.4	Policy-as-Code Direction	108
12.4.1	v2.0: Structured YAML Policy Blocks	108
12.4.2	v2.1+: OPA Integration Hooks	108
12.5	RBAC Model	109
12.5.1	Identity Establishment	109
12.5.2	Runbook-Level Access Control	110
12.5.3	Step-Level Role Restriction	110
12.5.4	Separation of Duties	110
12.6	Redaction	111
12.6.1	Pattern-Based Redaction	111
12.6.2	Explicit Field Marking (v2.1)	111
12.6.3	Redaction Guarantees	111
12.7	Audit Trail Requirements	111
12.7.1	Trace Integrity	112
12.7.2	Compliance Alignment	112
12.8	Extension-Contributed Policy	113
12.8.1	Extension Policy Rule Schema	113
13	Evidence, Tracing, and Resumption	114
13.1	Trace File Format	114
13.1.1	File Location and Naming	114
13.1.2	JSONL Envelope	115
13.1.3	Event Type Catalogue	115
	run/started	115
	step/started	116
	step/completed	116
	step/failed	117
	step/skipped	117

step/retrying	117
tool/invoked	117
tool/responded	118
manual/prompted	118
manual/completed	118
governance/blocked	119
governance/approved	119
run/completed	119
run/failed	120
checkpoint	120
13.1.4 Crash Safety	120
13.1.5 Tamper Evidence	121
13.2 Evidence Capture	121
13.2.1 What Constitutes Evidence	121
13.2.2 SHA256 Hashing	122
13.2.3 File Attachments	122
13.2.4 Evidence Retention	122
13.3 Run State Snapshots	123
13.3.1 Checkpoint Model	123
13.3.2 Atomic Snapshot Write	124
13.3.3 Minimum Resumption State	124
13.4 Run Resumption	125
13.4.1 The <code>gert exec -resume</code> Contract	125
13.4.2 Resumability by Step Type	125
13.4.3 Idempotency Guidance	126
13.4.4 Partial Step Resumption	126
13.5 Replay Mode	127
13.5.1 Overview	127
13.5.2 Scenario File Format	127
13.5.3 Replay Semantics	128
13.5.4 Determinism Requirement	128
13.6 OpenTelemetry Integration	128
13.6.1 Rationale	128
13.6.2 Span Hierarchy	129
13.6.3 Span Attributes	130
13.6.4 Relationship to JSONL Trace	130

13.6.5 Configuration	130
13.7 Implementation Notes for Go	131
14 Adapter Contracts	133
14.1 TODO	133
15 Input Provider Framework	134
15.1 Design Goals	134
15.2 Provider Definition Schema	134
15.3 Resolution Protocol	136
15.3.1 Resolution Flow	136
15.3.2 Resolution Request Envelope	137
15.3.3 Resolution Response Envelope	137
15.3.4 Resolution Error Handling	138
15.3.5 Resolution Caching	139
15.4 Built-in Providers	139
15.4.1 env	139
15.4.2 file	139
15.4.3 prompt	140
15.4.4 workspace	140
15.5 Provider Composition	141
15.6 Provider Lifecycle	141
16 Observability and Diagnostics	143
16.1 TODO	143
References	144

Chapter 1

Overview

1.1 Problem Statement

Operational procedures are the connective tissue of production engineering. When a service degrades, an incident fires, or a change must be applied under pressure, engineers rely on runbooks—step-by-step guides that encode institutional knowledge about how to respond. The research literature is unambiguous: *unstructured runbooks fail*. Static documents drift from reality, skip governance checks, and produce no audit evidence. Teams that operate at scale cannot afford a response that depends entirely on whether the on-call engineer remembers to follow the wiki [8, 19].

The core operational problem is threefold. First, runbooks must be **executable**: a document that cannot be run is a suggestion, not a procedure. Second, runbooks must be **governed**: every action that touches infrastructure must be bounded by policy—command allowlists, environment variable controls, output redaction, and role-based approval gates. Third, runbooks must be **traceable**: every execution must produce an immutable, auditable record that satisfies SOC 2, ISO 27001, and HIPAA audit requirements [20, 9, 23] without manual effort. Without all three properties, organizations face audit failures, runbook drift, and uncontrolled blast radius during incidents.

Gert is designed to solve exactly this problem: a governed, executable, debuggable runbook engine that produces full traceability and evidence by construction.

1.2 Why v2: What v1 Got Right, and What Constrained It

Gert v1 validated the core thesis. It delivered governed execution with command allowlists, environment variable blocking, output redaction, and per-step approval gates. It captured evidence—text, checklists, attachments with SHA256 hashes—and wrote an append-only

JSONL trace for every run. It provided deterministic replay via recorded scenarios, a full terminal UI (Bubble Tea), a VS Code extension over JSON-RPC 2.0, interactive debugging with a step-through REPL, and a `gert test` scenario runner. These are production-grade features that must be preserved in v2.

However, v1 accumulated architectural constraints that limit its evolution. The serve layer (`gert serve`) is tightly coupled to the runtime, making it difficult to add new adapters without modifying core logic. Schema versioning is brittle: `runbook/v0` and `runbook/v1` diverge with no formal compatibility contract. The tool and provider subsystems are functional but have no stable extension contract, making third-party tool development fragile. There is no saga or compensation pattern for rollback on failure, no parallel fan-out execution, no policy-as-code integration, and no OpenTelemetry support for distributed tracing—all of which are now industry-standard in comparable workflow orchestration systems [22, 4, 18].

These constraints are architectural, not cosmetic. A clean-break v2 is warranted.

1.3 Scope

Gert v2 is: a local-first, governed runbook execution engine with a stable component architecture, versioned extension contracts, a first-class governance and policy layer, full traceability by construction, and defined adapter interfaces for TUI, VS Code, and web clients. It ships as a single Go binary with an optional out-of-process extension runtime.

Gert v2 is not: a managed cloud execution service, a general-purpose workflow orchestrator, a CI/CD pipeline system, a SOAR platform, or a replacement for systems like Temporal, Argo, or PagerDuty Runbook Automation [22, 4, 17]. It does not provide a central policy server, a shared execution registry, or multi-tenant execution isolation.

1.4 Design Priorities

- **A core runtime isolated from presentation adapters.** The execution engine—parser, planner, state machine, governance enforcer, trace writer—must have no dependency on any presentation layer. TUI, VS Code, and web clients attach via a stable event subscription API; they observe execution but never own it. This isolation is the prerequisite for every other architectural goal.
- **Explicit and versioned contracts for schema, tools, and events.** Every interface that crosses a component boundary must be named, versioned, and documented. The

runbook schema carries an `apiVersion` field with formal compatibility guarantees. Tool definitions carry a version. The runtime event stream has a defined schema with ordered delivery guarantees. Breaking a contract requires a version increment, not a silent edit.

- **A secure extension runtime using out-of-process plugins.** Extensions run in isolated processes and are granted capabilities explicitly. No extension receives ambient authority. The capability model—what an extension may register, observe, or invoke—is a named, versioned list that the host enforces at admission time. This pattern follows the principle of least privilege endorsed by NIST SP 800-53 [13] and applied in production by systems such as Kubernetes admission controllers [7] and OPA Gatekeeper [14].

1.5 Document Roadmap

This document is organized as follows. §2 defines measurable goals and explicit non-goals, establishing scope boundaries for implementors. §?? describes the five primary components and the dependency rules between them. §?? specifies the v2 runbook schema, including versioning strategy, field inventory, and migration path from v1. §?? defines the extension runtime: capability taxonomy, handshake protocol, and trust model. §?? specifies the tool runtime: discovery, invocation, transport contracts, and governance integration. §?? defines the runtime event model: event schema, ordering guarantees, delivery semantics, and replay contract. §?? specifies the security and trust model: threat model, policy enforcement, supply chain controls, and audit requirements. §?? defines the testing and acceptance strategy: contract test categories, coverage requirements, and acceptance criteria. §10 documents open architectural questions with options and resolution criteria.

1.6 Success Definition

Gert v2 is complete when: (1) a v1 runbook can be migrated to v2 schema via `gert migrate` and executed without manual fixup; (2) the TUI, VS Code extension, and a third-party adapter can all be built against the published event and API contracts without importing internal packages; (3) the governance layer—allowlists, denylists, env blocking, redaction, approval gates—is enforced for every execution mode including extension-invoked runs; (4) every execution produces an append-only, crash-safe JSONL trace that is structurally valid

against the v2 trace schema; and (5) **gert test** passes a scenario suite covering all step types, governance rules, and evidence capture cases.

Chapter 2

Goals and Non-Goals

2.1 Goals

The following goals are the binding commitments of the v2 design. Where possible, each goal is stated as a verifiable criterion rather than an aspiration.

- G1. Preserve and formalize the governance layer.** Every capability that made v1 operationally useful—command allowlists and denylists, environment variable blocking, output redaction with regex patterns, and per-step role-based approval gates—must be present in v2 with identical or stronger semantics. Governance is gert’s core differentiator [17, 2]: approval gates integrated directly into step definitions. v2 must additionally support RBAC scoping on runbook execution: which identities may run which runbooks, and under what conditions.
- G2. Preserve the append-only, crash-safe, tamper-evident trace.** The v2 execution trace must be an append-only JSONL file. Each line is a self-describing JSON object with a unique event ID (UUID), ISO 8601 UTC timestamp [11], actor identity, step ID, and structured payload. The file must be crash-safe (partial writes must not corrupt prior entries) and must support optional cryptographic signing to satisfy SOC 2 and ISO 27001 (A.12.4) audit requirements [20, 9]. This is a launch blocker: no trace, no compliance.
- G3. Preserve evidence capture, deterministic replay, TUI, VS Code extension, and gert test.** The following v1 capabilities must be functionally preserved in v2: evidence capture with text, checklists, and SHA256-hashed attachments; scenario-based deterministic replay (`-mode replay`); the Bubble Tea terminal UI [5]; the VS Code extension powered by the JSON-RPC server [10, 12]; and the `gert test` scenario

runner with its assertion engine. These features represent validated operational value. Their absence in v2 is a regression, not a simplification.

- G4. Establish stable, versioned extension contracts.** Tool definitions (`.tool.yaml`), input providers (`.provider.yaml`), and out-of-process extensions must bind to named, versioned contracts. A host version bump that breaks an extension contract requires a major version increment and a migration path. The capability taxonomy—what an extension may register, observe, and invoke—must be a published, enumerated list, not an implicit set of permissions inferred from code. This pattern is established by VS Code’s extension API [12] and Kubernetes admission controller design [7].
- G5. Introduce saga and compensation for multi-step rollback.** v2 must support explicit compensation registration: when a forward step completes, the runbook may declare a compensating action to execute in reverse order on failure. This is the industry-standard saga pattern, implemented in production by Temporal [22] and AWS Step Functions [1]. Without compensation, partially-executed runbooks that modify infrastructure leave systems in undefined intermediate states.
- G6. Integrate OpenTelemetry for distributed tracing.** Every runbook execution must emit OpenTelemetry spans [16]: one root span per run, one child span per step, with W3C Trace Context [24] (`traceparent` / `tracestate`) propagated to tool invocations. Correlation IDs must appear in both the JSONL trace and the OTEL span attributes, so that run traces can be joined with infrastructure observability data in Jaeger [6], Zipkin [26], or any OTLP-compatible backend. This is the current industry standard for operational visibility.
- G7. Decouple adapters from core via a typed event API.** The TUI, VS Code extension, and any future web adapter must attach to the core exclusively via a typed event subscription interface. No adapter may import internal runtime packages or call runtime methods directly. The event schema must be versioned: adapters declare which schema version they consume, and the host rejects incompatible connections. This constraint is necessary for gert to support concurrent adapters (e.g., TUI and VS Code running simultaneously against a single run).
- G8. Establish migration compatibility for v1 runbooks.** The command `gert migrate` (v2 edition) must produce a valid `runbook/v2` document from any well-formed `runbook/v0` or `runbook/v1` input with no manual fixup required for standard runbooks. The v2 runtime should additionally support a compatibility execution

mode that accepts v1 runbooks without prior migration, so that existing users are not blocked from adopting v2 incrementally.

2.2 Non-Goals

The following are explicitly outside the scope of gert v2. These boundaries are feature-scoped, not process-scoped, and are intended to prevent scope creep during implementation.

- NG1. Gert v2 does not provide a managed cloud execution service.** There is no gert-as-a-service, no central execution registry, and no SaaS offering. Gert v2 is a local-first binary. Cloud scheduling, multi-tenant isolation, and remote execution orchestration are concerns for a hypothetical v3 or a separate product.
- NG2. Gert v2 does not implement a general-purpose workflow orchestrator.** Gert is not Temporal, Argo, or Prefect [22, 4, 18]. It does not support distributed workers, fan-out parallelism across remote nodes, or arbitrary DAG execution. Parallel execution within a single local run is a stretch goal; cross-host parallelism is explicitly out of scope.
- NG3. Gert v2 does not ship a built-in policy server.** While v2 defines governance semantics and may integrate policy-as-code hooks [15], it does not bundle or operate an OPA server. Policy evaluation happens at the host process level. Central policy distribution, policy bundles over HTTP, and multi-host policy synchronization are out of scope.
- NG4. Gert v2 does not define an extension marketplace or signing authority.** v2 will define an extension signing *mechanism* (a verifiable public key model), but it will not operate a registry, curate extensions, or act as a certificate authority. Extension distribution is the responsibility of the extension author.
- NG5. Gert v2 does not guarantee binary backward compatibility with v1 internal packages.** v2 is a clean-break architecture. The internal Go package structure, private interfaces, and runtime internals are not subject to backward compatibility guarantees. Only the documented public contracts—the runbook schema, the tool schema, the JSON-RPC API surface, the extension protocol, and the JSONL trace schema—carry compatibility commitments.

Chapter 3

Architecture

This chapter defines the v2 system architecture: the five primary components, their interfaces and invariants, the complete data flow from runbook invocation to trace archive, the execution lifecycle, the concurrency model, and how the `gert serve` JSON-RPC layer integrates as a first-class adapter.

The single most important architectural rule is: **dependencies flow inward toward the Core Domain**. The Runtime Core knows nothing about TUI, VS Code, or JSON-RPC. Adapters know nothing about execution semantics. Every component boundary is enforced by a Go interface; no component imports a package that is “above” it in the dependency graph.

3.1 Primary Components

3.1.1 Parser

Responsibility. The Parser owns the transformation of a raw YAML document into a validated, semantically-resolved `ParsedRunbook` value. It does *not* own execution, planning, tool discovery, or input resolution. It validates schema structure and surface-level semantic rules (duplicate step IDs, unknown step types, malformed Go template expressions) but intentionally defers deep semantic validation (e.g., whether a `from:` binding can actually be resolved) to later pipeline stages.

Interface.

```
// Parser transforms raw YAML bytes into a validated ParsedRunbook.
type Parser interface {
    // Parse reads YAML from r, validates against the v2 JSON Schema,
```

```

    // applies structural semantic checks, and returns a ParsedRunbook.
    // Errors carry structured diagnostics (file, line, column, message).
    Parse(ctx context.Context, r io.Reader, opts ParseOptions) (*ParsedRunbook, error)
}

type ParseOptions struct {
    // SchemaVersion overrides the apiVersion field for compatibility shims.
    SchemaVersion string
    // StrictMode causes deprecation warnings to be surfaced as errors.
    StrictMode bool
}

// ParsedRunbook is an immutable, fully-validated representation of a runbook.
// All fields have been structurally validated. Templates are not yet evaluated.
type ParsedRunbook struct {
    Meta          RunbookMeta
    Governance GovernancePolicy // nil if no governance block
    Steps         []ParsedStep
    Imports       map[string]string // alias -> path (not yet resolved)
    Tools         []string          // tool names (not yet discovered)
    Schema        SchemaRef          // version and $schema URL
}

```

Dependencies. The Parser depends only on the JSON Schema artifact (a static embedded file) and the expression evaluator (for template syntax validation). It has no runtime dependencies.

Key Invariants.

- A `ParsedRunbook` returned without error is structurally valid against the v2 JSON Schema.
- All step IDs are unique within a `ParsedRunbook`.
- No I/O occurs during parsing (file imports are recorded as paths, not resolved).
- Parsing is pure and deterministic: the same input always produces the same output.

3.1.2 Planner

Responsibility. The Planner transforms a `ParsedRunbook` into an `ExecutionPlan`: a resolved, ordered, dependency-annotated representation of every step the runtime will execute. The Planner resolves imports (recursively parsing and inlining invoked runbooks), discovers and validates tool definitions, resolves input provider bindings to their declared sources, and produces the static step ordering. It does *not* evaluate dynamic conditions (branch predicates, iterate count expressions) — those are runtime concerns.

What is an `ExecutionPlan`? An `ExecutionPlan` is a *flat, ordered list* of resolved steps with pre-computed metadata, not a DAG. Branch logic and iteration are left as runtime decisions because they depend on captured variables evaluated during execution. The plan encodes *which steps exist* and *what preconditions they carry*, not which steps will actually run. This design keeps the planner pure and the runtime authoritative over control flow.

```
type Planner interface {
    // Plan resolves imports, discovers tools, validates bindings,
    // and returns a fully-resolved ExecutionPlan ready for the runtime.
    Plan(ctx context.Context, rb *ParsedRunbook, opts PlanOptions) (*ExecutionPlan, error)
}

type PlanOptions struct {
    BaseDir      string           // working directory for import resolution
    ToolDirs     []string          // directories to search for .tool.yaml files
    ProviderDir  string           // directory for .provider.yaml files
    Vars         map[string]string // CLI-supplied variable overrides
}

// ExecutionPlan is an immutable, fully-resolved plan for a single run.
type ExecutionPlan struct {
    RunID        string
    RunbookPath  string
    Steps        []ResolvedStep // ordered; includes inlined invoke steps
    Tools        map[string]*ToolDef // name -> resolved tool definition
    Providers    map[string]*ProviderDef // binding name -> provider
    Governance   *GovernancePolicy
    Metadata     PlanMetadata
}
```

```
}

// ResolvedStep is a single step in the plan, fully resolved.
type ResolvedStep struct {
    ID      string
    Kind    StepKind    // cli, manual, tool, invoke, branch, iterate
    Spec    StepSpec     // kind-specific parameters (resolved, not templated)
    Contract *contract.Contract // effects/reads/writes/idempotent/deterministic
    Depth   int           // chain depth (0 = root runbook)
    Origin  string        // source runbook path (for inlined invoke steps)
}
```

Dependencies. The Planner depends on the Parser (for import resolution), the Tool Registry (for `.tool.yaml` discovery), and the Provider Registry (for `.provider.yaml` discovery). It must not depend on the Runtime Core, Extension Host, or any adapter.

Key Invariants.

- An `ExecutionPlan` returned without error contains only resolvable steps.
- All tool names referenced by steps appear in `ExecutionPlan.Tools`.
- Import graphs are acyclic; the Planner detects and rejects cycles.
- All `from:` bindings that declare a provider appear in `ExecutionPlan.Providers`.
- The Planner does not execute or side-effect; it is safe to plan without running.

3.1.3 Runtime Core

Responsibility. The Runtime Core owns the execution loop: advancing through an `ExecutionPlan` step by step, evaluating templates and branch predicates, dispatching tool invocations, collecting evidence, writing the append-only trace, enforcing governance pre-flight checks, emitting structured events, and managing the mutable run state. It does *not* own user presentation, serialization formats, or network transport.

Interface.

```
type Runtime interface {
    // Start initializes a run from a plan and returns a RunHandle.
    // The run does not advance until Next is called.
    Start(ctx context.Context, plan *ExecutionPlan, opts RunOptions) (RunHandle, error)

    // Resume restores a previously checkpointed run from the run store.
    Resume(ctx context.Context, runID string, opts RunOptions) (RunHandle, error)
}

type RunHandle interface {
    // Next advances the run by one step. Blocks until the step completes
    // (or is paused waiting for evidence/approval).
    // Returns io.EOF when the run has no more steps.
    Next(ctx context.Context) (*StepResult, error)

    // Approve records approval for the current step's approval gate.
    Approve(ctx context.Context, decision ApprovalDecision) error

    // SubmitEvidence records evidence for the current manual step.
    SubmitEvidence(ctx context.Context, stepID string, ev map[string]*EvidenceValue) error

    // Cancel requests a graceful stop of the run.
    Cancel(ctx context.Context, reason string) error

    // State returns the current mutable run state (read-only snapshot).
    State() RunState

    // Events returns a channel of structured events emitted during execution.
    Events() <-chan Event
}

type RunOptions struct {
    Mode      RunMode      // real, dry-run, replay
    Actor     string       // --as identity
```

```

    Vars          map[string]string // variable overrides
    ScenarioDir   string             // replay scenario directory
    Store         RunStore           // durable store for checkpointing
    OnEvent       func(Event)        // synchronous event hook (nil = use Events channel)
}

```

Dependencies. The Runtime Core depends on the Governance Engine, Tool Runtime (for tool invocation), Expression Evaluator (for templates and branch predicates), Evidence store, and Trace Writer. It does not depend on the Extension Host, API/Adapter Layer, or any adapter.

Key Invariants.

- The trace is append-only: no event is ever deleted or modified after being written.
- Governance pre-flight is enforced before every `cli` and `tool` step dispatch.
- Redaction is applied to all captured output before it is written to the trace.
- The run state is checkpointed after every step completion.
- Events are emitted in a total order that matches step execution order.
- The runtime is re-entrant per run: multiple callers may call `Next` sequentially (but not concurrently) on the same `RunHandle`.

3.1.4 Extension Host

Responsibility. The Extension Host manages the lifecycle of out-of-process extensions: discovery, launch, capability negotiation, RPC dispatch, sandboxing, and graceful shutdown. It exposes extension-contributed tools and providers to the Tool Runtime and Provider Registry. It does *not* execute runbook steps directly and does not own the extension communication protocol (that belongs to the extension RPC layer, defined in §04).

Interface.

```

type ExtensionHost interface {
    // Load discovers and launches all extensions declared in the project manifest.

```

```
Load(ctx context.Context, manifest *ProjectManifest) error

// Shutdown gracefully stops all running extensions.
Shutdown(ctx context.Context) error

// ContributedTools returns all tools contributed by loaded extensions.
ContributedTools() []*ToolDef

// ContributedProviders returns all providers contributed by loaded extensions.
ContributedProviders() []*ProviderDef

// ContributedPolicyRules returns governance rules contributed by extensions.
// These compose with host policy; host policy takes precedence on conflict.
ContributedPolicyRules() []PolicyRule
}
```

Dependencies. The Extension Host depends on the extension manifest schema and the extension RPC transport (stdio JSON-RPC, as specified in §04). It does not depend on the Runtime Core or any adapter.

Key Invariants.

- No extension operates without an explicit capability grant from the host.
- Extension crashes do not crash the host process.
- The Extension Host enforces timeout and cancellation on all RPC calls to extensions.
- Extension-contributed policy rules are additive; they can only *restrict*, not *relax*, host-defined policy.

3.1.5 API / Adapter Layer

Responsibility. The API/Adapter Layer is the boundary between the Runtime Core and all external consumers (TUI, VS Code extension, web clients, CI pipelines). It translates the runtime's event stream and `RunHandle` contract into transport-specific protocols. Each adapter is a thin, stateless renderer: it subscribes to the event stream, presents state to the

user, and forwards user input (approvals, evidence, cancellation) back to the `RunHandle`. Adapters own *no* execution logic.

Interface.

```
// Adapter is the interface every consumer of the Runtime Core must implement.
type Adapter interface {
    // Attach binds the adapter to a RunHandle before the first Next call.
    Attach(handle RunHandle) error

    // Run blocks until the run completes or is cancelled. It is responsible
    // for draining Events(), rendering output, and forwarding user input.
    Run(ctx context.Context) error
}

// The gert serve JSON-RPC layer implements Adapter over stdio.
// The TUI (Bubble Tea) implements Adapter over a terminal.
// Future: a web adapter implements Adapter over WebSocket.
```

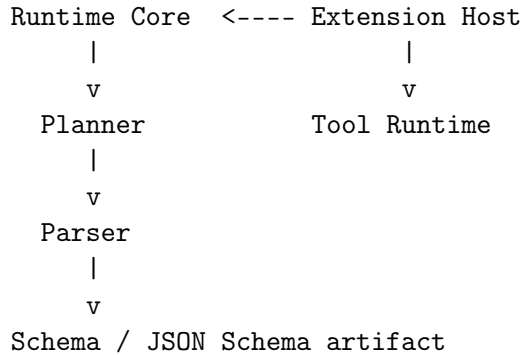
Dependencies. Adapters depend on the `RunHandle` and `Event` types from the Runtime Core. They must not import any package from the Parser, Planner, or Extension Host.

Key Invariants.

- An adapter may not call `Next` more than once concurrently.
- An adapter may not modify run state directly; all state changes go through `RunHandle`.
- Adapters must handle the case where the event channel is closed (run ended) without blocking.
- An adapter failure must not corrupt the run trace or run state.

Dependency Direction Summary.

```
Adapter Layer
|
v
```



No upward arrow exists. The Parser knows nothing about the Runtime. The Runtime knows nothing about adapters. The Extension Host is a peer of the Runtime Core, contributing tools and policy rules but not owning execution.

3.2 Data Flow

This section traces the complete path from runbook invocation to trace archive, identifying exactly where each responsibility is exercised.

3.2.1 Phase 1: Parse

The user invokes `gert exec runbook.yaml -as eng@corp.com`. The CLI layer reads the YAML file and calls `Parser.Parse()`. The Parser validates the document against the embedded v2 JSON Schema, checks structural semantic rules, and returns a `ParsedRunbook`. If validation fails, structured diagnostics are returned and execution does not proceed.

3.2.2 Phase 2: Plan

The `ParsedRunbook` is passed to `Planner.Plan()`. The Planner:

1. Resolves all `imports:` recursively (parse $\rightarrow$ plan each imported runbook, inline its steps with incremented `Depth`).
2. Discovers `.tool.yaml` files for each named tool.
3. Resolves `from:` input bindings to their declared providers.

4. Validates that all step references (branch targets, iterate bodies) are valid step IDs.
5. Constructs and returns an `ExecutionPlan`.

Governance policy from the runbook's `meta.governance` block is extracted and embedded in the `ExecutionPlan` for runtime enforcement.

3.2.3 Phase 3: Run Initialization

`Runtime.Start(plan, opts)` is called. The runtime:

1. Generates a `RunID` (format: `YYYYMMDDTHH:mm:ss-xxxxxxx`).
2. Creates the run directory: `.runbook/runs/<runID>/`.
3. Opens the append-only trace file: `.runbook/runs/<runID>/trace.jsonl`.
4. Writes the `run/started` trace event (runbook path, actor, mode, plan hash).
5. Initializes the `GovernanceEngine` from the plan's governance policy.
6. Compiles redaction rules from the governance policy.
7. Returns a `RunHandle`; the run is now paused at step 0.

3.2.4 Phase 4: Step Execution Loop

The adapter calls `RunHandle.Next(ctx)` in a loop until `io.EOF`. For each step, the runtime executes the following sequence:

Step Pre-flight (all step types)

1. Write `step/started` trace event.
2. Emit `StepStarted` runtime event to the event channel.
3. Evaluate template expressions in the step spec against current run variables.

Governance Pre-flight (cli and tool steps)

1. **Command check:** `GovernanceEngine.CheckCommand(argv[0])` evaluates the denylist (deny takes precedence) and then the allowlist. A violation writes a `governance/commandBlocked` trace event and returns an error; the step fails.
2. **Env var check:** `GovernanceEngine.FilterEnvVars()` removes variables matching `deny_env_vars` patterns. Blocked variable names are written to the `governance/envVarBlocked` trace event.
3. **Contract check:** The step's `Contract.Risk()` is evaluated. Steps with `RiskCritical` that do not have an explicit approval gate declaration in the plan emit a `governance/contractWarning` event (not a hard failure in v2.0; upgraded to error in strict mode).
4. **Approval gate:** If the step declares `approvals.min > 0`, the runtime pauses and emits an `ApprovalRequired` event. Execution blocks at this step until `RunHandle.Approve()` is called with sufficient approvals. See §12.3 (Governance chapter) for the full approval protocol.

Dispatch (step-type-specific)

- **cli:** Fork a subprocess. Capture stdout/stderr. Apply redaction rules to captured output before storing in run variables or writing to trace.
- **tool:** Call `ToolRuntime.Invoke(toolName, input)`. The tool runtime dispatches to the appropriate transport (stdio, JSON-RPC, MCP). Redaction applied to returned output.
- **manual:** Emit `EvidenceRequired` event. Block until `SubmitEvidence()` is called. SHA256-hash each evidence attachment.
- **invoke:** Inlined at plan time; at runtime the invoke body steps are already present in the plan at `Depth+1`. No special dispatch required.
- **branch:** Evaluate branch predicates (expression evaluator) against current variables. Select the matching branch arm. Write `event/branchResolved` trace event. Advance step cursor to the first step of the resolved arm.
- **iterate:** Evaluate the iteration expression. Write `event/iteratePassStart`. Execute body steps. Write `event/iteratePassEnd`. Repeat until condition is false.

Step Post-flight (all step types)

1. Apply outcome evaluation: test `when:` expressions; record `OutcomeRecord`.
2. Write captured variables to run state.
3. Checkpoint run state to `.runbook/runs/<runID>/snapshots/<stepID>.json`.
4. Write `step/completed` trace event (outcome, captures, elapsed time).
5. Emit `StepCompleted` runtime event.

3.2.5 Phase 5: Run Completion

When `Next` returns `io.EOF`:

1. Write `run/completed` trace event (final outcome, step counts, elapsed time).
2. Flush and sync the trace file.
3. Compute SHA256 of the trace file; write `run/evidenceHash` record.
4. Optionally archive the run directory (compress to `.runbook/archive/`).
5. Close the event channel.

3.3 Lifecycle Model

3.3.1 Run Start

A run starts in one of two ways:

1. **Direct invocation:** The CLI or test harness calls `Runtime.Start()` directly. This is the path for `gert exec`, `gert tui`, `gert debug`, and `gert test`.
2. **RPC invocation:** A client sends `exec/start` to the JSON-RPC server (`gert serve`). The server calls `Runtime.Start()` on behalf of the client and returns the `RunID` in the response. Subsequent step advancement is driven by `exec/next` RPC calls.

3.3.2 Step Advancement

Step advancement is *always client-driven*. The runtime does not auto-advance. After **Start**, the client calls **Next** to execute each step in sequence. This applies to all adapters, including headless CI mode. The only exception is the **-auto** flag, which causes the CLI adapter to call **Next** in a tight loop, producing the appearance of automatic advancement while retaining the single-step contract.

3.3.3 Pause and Resume

A run is *paused* when **Next** returns a non-terminal result that requires external input: evidence submission (**manual step**) or approval (**approval gate**). The run remains paused until the required input is provided via **SubmitEvidence** or **Approve**. The adapter is responsible for presenting the pause to the user.

The checkpoint written after each completed step is sufficient to resume a run after a process restart. The checkpoint records: completed step IDs, current variable bindings, current evidence values, and the trace sequence number. Resumption is initiated by calling **Runtime.Resume(runID)** from any adapter.

3.3.4 Cancellation

RunHandle.Cancel(ctx, reason) initiates graceful cancellation:

1. The currently-executing subprocess (if any) receives **SIGTERM** followed by **SIGKILL** after a 5-second grace period.
2. The runtime writes a **run/cancelled** trace event with the reason and actor.
3. Subsequent **Next** calls return **ErrRunCancelled**.
4. The run state is checkpointed as **cancelled** (resumable).

The cancellation contract does not invoke compensation handlers in v2.0. Saga/compensation (stepping backward through a compensation chain on failure) is deferred to v2.1 as specified in §09 (Open Questions).

3.3.5 Crash Recovery

If the `gert` process crashes mid-run, the trace and checkpoint files remain on disk in a consistent state due to the append-only, sync-after-write trace discipline. On restart, the user (or the VS Code extension) can call `Runtime.Resume(runID)` to continue from the last completed step. The runtime reads the checkpoint to reconstruct run state. Steps already recorded as completed in the trace are not re-executed.

3.4 Concurrency Model

3.4.1 Per-Process Run Isolation

The v2 runtime is **single-run-per-process**. Each `gert exec` invocation is a separate OS process. `gert serve` is the exception: it is a long-running process that may host multiple sequential runs (one at a time) for a single VS Code extension session. Concurrent runs within a single `gert serve` instance are explicitly out of scope for v2.0 (see §09).

Rationale. Governance enforcement, file-level trace writes, and subprocess management are dramatically simpler with one run per process. Process isolation also provides a natural crash boundary: a failed run cannot corrupt the state of a concurrent run.

3.4.2 Goroutine Model

Within a single run, the concurrency model is:

- **One goroutine per run:** the step execution loop runs on a single goroutine. Steps execute sequentially; there is no goroutine-per-step parallelism in v2.0.
- **Event fan-out goroutine:** a second goroutine reads from an internal event buffer and writes to the `Events()` channel. This decouples event emission (which must be synchronous with step execution) from event consumption (which may be slow, e.g., a WebSocket write).
- **Subprocess goroutine:** when a `cli` step spawns a subprocess, a goroutine is used to drain stdout/stderr concurrently, preventing deadlock on full pipe buffers.
- **Tool RPC goroutine:** tool invocations over JSON-RPC or MCP use a goroutine per call, managed by the Tool Runtime's connection pool.

3.4.3 Thread Safety at Component Boundaries

- `RunHandle` is **not safe for concurrent use**. The adapter must serialize all calls. `gert serve` enforces this with a per-session mutex.
- The `Events()` channel is safe for a single consumer goroutine.
- The `GovernanceEngine` is **read-only after construction** and therefore safe for concurrent read access (relevant if future versions support parallel steps).
- The Trace Writer is protected by an internal mutex; write calls are safe to issue from any goroutine within the run.

3.5 `gert serve` Integration

`gert serve` is the JSON-RPC 2.0 server that powers the VS Code extension, the MCP tools layer, and any future web client. In the v2 architecture it is classified as an **adapter in the API/Adapter Layer**.

3.5.1 Architectural Position

`gert serve` is a process that:

1. Starts a long-lived stdio JSON-RPC 2.0 listener (newline-delimited JSON over stdin/stdout).
2. On receiving `exec/start`, calls `Runtime.Start()` and stores the resulting `RunHandle` in a session map (keyed by `RunID`).
3. On receiving `exec/next`, calls `RunHandle.Next()` and returns the `StepResult` as the RPC response.
4. Drains the `Events()` channel in a background goroutine and forwards each event as a JSON-RPC notification to the client.
5. On receiving `exec/approve`, `exec/submitEvidence`, or `exec/cancel`, dispatches to the corresponding `RunHandle` method.

3.5.2 Event Forwarding

Every event emitted by the Runtime Core is forwarded to the client as a JSON-RPC *notification* (no `id` field) with method `event/eventType`. The payload is the event's JSON representation. The VS Code extension subscribes to these notifications to update its UI without polling.

Events forwarded over WebSocket (for a future web adapter) follow the same schema; only the transport changes.

3.5.3 Crash Recovery in gert serve

If the VS Code extension reconnects after a crash (e.g., the gert process was killed), `gert serve` exposes `exec/list` and `exec/resume` methods. The extension discovers the interrupted run, calls `exec/resume`, and receives a new `RunHandle` over the same session. The trace and checkpoint on disk ensure no steps are lost.

3.5.4 RPC Method Summary

<code>exec/start</code>	→ Start a new run (returns <code>RunID</code> , first <code>StepResult</code>)
<code>exec/next</code>	→ Advance run by one step (returns <code>StepResult</code>)
<code>exec/approve</code>	→ Submit approval decision for current gate
<code>exec/submitEvidence</code>	→ Submit evidence for current manual step
<code>exec/cancel</code>	→ Cancel the active run
<code>exec/list</code>	→ List all runs for this workspace (active + completed)
<code>exec/resume</code>	→ Resume a previously interrupted run
<code>runbook/validate</code>	→ Validate a runbook YAML (parse + plan, no exec)
<code>runbook/diagram</code>	→ Generate a Mermaid or ASCII diagram
<code>event/<type></code>	→ Notification from server to client (not a request)

Chapter 4

Schema vNext

This chapter is the normative specification for all gert v2 data structures: runbook documents, tool definitions, and provider definitions. It supersedes all v1 schema documentation and defines the contracts that Brian’s parser, Ken’s runtime, and the VS Code extension must implement.

The design philosophy is *minimal core, extensible periphery*: the core language fields that the runtime must understand are small, typed, and stable; everything else is declared as an extension with an explicit namespace. Where v1 accumulated surface area organically, v2 draws a hard boundary and enforces it at parse time.

4.1 Schema Identity and Versioning

4.1.1 apiVersion Values

Every gert document carries a mandatory **apiVersion** field that identifies both the document kind and the schema version. The v2 values are:

Document kind	v1 value	v2 value
Runbook	runbook/v1	runbook/v2
Tool definition	tool/v0	tool/v2
Provider definition	provider/v0	provider/v2

The format is `<kind>/<major>`. Minor and patch changes do not alter the **apiVersion** string; they are tracked by the schema artifact version (see §4.9).

4.1.2 Self-Description via \$schema

Following JSON Schema [25] best practice and the pattern used by GitHub Actions, Argo Workflows, and Kubernetes CRDs, every v2 document *should* carry a `$schema` field pointing to the canonical JSON Schema URL:

```
$schema: "https://schemas.gert.dev/runbook/v2.json"
apiVersion: runbook/v2
id: service-health
name: Service Health Check
...
```

The `$schema` field is optional at runtime (the runtime uses `apiVersion` for dispatch) but is *strongly recommended* because:

- VS Code and JetBrains IDEs provide schema-backed auto-completion and inline validation when `$schema` is present.
- The schema URL carries an implicit version contract: updating the URL updates the validation rules visible to tooling.
- It enables offline validation with any JSON Schema Draft 2020-12 validator (ajv, jsonschema, check-jsonschema).

Schema URLs for each document kind:

```
https://schemas.gert.dev/runbook/v2.json
https://schemas.gert.dev/tool/v2.json
https://schemas.gert.dev/provider/v2.json
```

4.1.3 Versioning Strategy

gert schema versioning follows a simplified semantic versioning model applied to the `apiVersion` major component:

- **Major increment** (e.g. `runbook/v1` → `runbook/v2`): A breaking change. The parser *must not* silently accept the old format. Automated migration is provided via `gert migrate`.

- **Minor/patch changes within a major:** Additive, backward-compatible. New optional fields may be added. Existing fields may not be removed or renamed. The JSON Schema artifact version tracks these with a semver suffix (e.g. `runbook/v2.json` at artifact version `2.1.0`).

What constitutes a breaking change A breaking change is any of:

- Renaming or removing a required field.
- Narrowing an existing enum (removing a valid value).
- Changing a field type (e.g. `string` $\rightarrow$ `object`).
- Changing the meaning of a field such that existing runbooks produce different runtime behavior.
- Removing a step type.

Adding new optional fields, adding new enum values, and loosening existing constraints are *not* breaking changes.

4.1.4 Compatibility Contract: v1 → v2

v1 field	v2 status	Notes
apiVersion: runbook/v0	Removed	Handled by <code>gert migrate</code>
apiVersion: runbook/v1	Deprecated	Compatibility mode in v2 parser
meta.name	Promoted	Becomes top-level <code>name</code>
meta.kind	Promoted	Becomes top-level <code>kind</code>
meta.description	Promoted	Becomes top-level <code>description</code>
meta.vars	Retained	Moved to top-level <code>vars</code>
meta.inputs	Retained	Moved to top-level <code>inputs</code>
meta.governance	Retained	Moved to top-level <code>governance</code>
meta.prose	Retained	Moved to top-level <code>prose</code>
meta.defaults	Retained	Moved to top-level <code>defaults</code>
imports	Retained	Unchanged
tools	Replaced	Replaced by <code>toolRefs</code> (typed)
flow	Retained	Canonical execution root
tree	Removed	Use <code>flow</code>
steps	Removed	Flat step list was never canonical
step.type: collector	Renamed	Becomes <code>type: manual</code>
step.type: router	Renamed	Becomes <code>type: end</code>
step.type: choice	Retained	Unchanged
step.type: noop	Removed	Use <code>when: guard</code> on any step
step.type: extension	Retained	Unchanged
step.with.argv	Deprecated	Use <code>type: cli</code> with <code>command:</code>
step.assertions	Deprecated	Use <code>step.type: assert</code>

The v2 parser reads `runbook/v1` documents in **compatibility mode**: all v1 field paths are accepted and silently normalized to their v2 equivalents before validation. Running `gert migrate` permanently rewrites the file to v2 canonical form. v2 will not support `runbook/v0`; documents at that version must be migrated to v1 first or directly to v2 via a two-stage migration.

4.1.5 gert migrate Behavior

```
# Dry-run preview
gert migrate --dry-run runbook.yaml
```

```
# In-place rewrite
gert migrate runbook.yaml
```

```
# Migrate a whole directory
gert migrate ./runbooks/
```

Migration rules applied automatically:

1. Rewrite `apiVersion` to `runbook/v2`.
2. Add `$schema` field.
3. Promote `meta.name`, `meta.kind`, `meta.description` to top-level.
4. Move remaining `meta.*` fields to their v2 top-level equivalents.
5. Rename `tree:` to `flow:`.
6. Rename `step.type: collector` → `manual`, `step.type: router` → `end`.
7. Rewrite `tools: [name]` shorthand to `toolRefs:` typed declarations.
8. Add `id:` to runbook if absent (derived from `name`).

4.2 Core Runbook Fields (v2)

4.2.1 Top-Level Field Inventory

The v2 runbook document has the following top-level fields:

Field	Type	Required	Description
<code>\$schema</code>	string	No	JSON Schema URL (recommended)
<code>apiVersion</code>	string	Yes	Must be <code>runbook/v2</code>
<code>id</code>	string	Yes	Unique machine identifier (slug)
<code>name</code>	string	Yes	Human-readable display name
<code>kind</code>	enum	No	<code>mitigation</code> <code>reference</code> <code>composable</code> <code>rca</code>
<code>description</code>	string	No	Short prose summary
<code>vars</code>	map	No	Default variable values
<code>inputs</code>	map	No	Input declarations (§4.3)
<code>outputs</code>	map	No	Output declarations (§4.3)
<code>toolRefs</code>	array	No	Tool references (§4.2.2)
<code>imports</code>	map	No	Named runbook imports
<code>defaults</code>	object	No	Default step settings
<code>governance</code>	object	No	Governance policy
<code>prose</code>	object	No	Human-readable TSG sections
<code>metadata</code>	map	No	Arbitrary key-value annotations
<code>flow</code>	array	Yes	Execution tree (TreeNode array)

id field The `id` is a slug: lowercase alphanumeric with hyphens, matching `^[a-z][a-z0-9-]*[a-z0-9]$`. It uniquely identifies a runbook within a project and is used by the `invoke` step and the trace record. It must be stable across renames; changing it is a breaking change for any runbook that invokes it by `id`.

kind values

- **mitigation** — Incident response procedure; governance defaults to `require-approval` for destructive effects.
- **reference** — Documentation-first runbook; dry-run by default.
- **composable** — Sub-runbook intended for invoke from other runbooks; not directly executable from CLI.
- **rca** — Root-cause analysis template.

4.2.2 toolRefs

In v1, tools were declared as a flat string array (`tools: [curl, nslookup]`). In v2, `toolRefs` is a typed array that allows aliasing and explicit path overrides:

```
toolRefs:
  - name: curl
  - name: nslookup
  - name: k8s
    path: ./tools/k8s.tool.yaml # override default discovery
    alias: kubect1-wrapper      # optional local alias
```

The default discovery path for `name: foo` is `tools/foo.tool.yaml` relative to the runbook file. The `path` field overrides this. The `alias` field lets the runbook reference the tool by a local name that differs from the tool's declared `meta.name`.

4.2.3 Minimal Valid v2 Runbook

```
$schema: "https://schemas.gert.dev/runbook/v2.json"
apiVersion: runbook/v2
id: hello-world
name: Hello World
```

```
flow:
  - step:
      id: greet
      type: cli
      title: Print greeting
      command: echo
      args: ["Hello, world!"]
      capture:
        output: stdout
  - step:
      id: done
      type: end
      title: Complete
      outcome:
```

```
category: resolved
code: success
```

4.2.4 Complete Annotated Example

```
$schema: "https://schemas.gert.dev/runbook/v2.json"
apiVersion: runbook/v2
id: service-health-check
name: Service Health Check
kind: mitigation
description: |
  Validate DNS resolution and HTTP endpoint reachability
  for a named service. Escalates on failure.

vars:
  default_timeout: "30s"

inputs:
  hostname:
    type: string
    required: true
    description: "Hostname to check"
    from: prompt
  incident_id:
    type: string
    required: false
    description: "Incident tracking ID"
    from: incident.id      # provider-resolved

outputs:
  health_status:
    type: string
    description: "Final health assessment"
    from: captures.http_result

toolRefs:
```

```
- name: curl
- name: nslookup

governance:
  rules:
    - effects: ["network.probe"]
      action: allow
    - effects: ["service.restart"]
      action: require-approval
      min_approvers: 2
  redact:
    - pattern: "(?i)Authorization:\\s*\\S+"
      replace: "Authorization: <redacted>"

defaults:
  timeout: "60s"

prose:
  safety: |
    Run this runbook only during an active incident. Do not
    loop - each execution creates a trace record.
  prerequisites: "Requires curl >= 7.80 and nslookup on PATH."

metadata:
  team: platform-sre
  runbook-url: "https://wiki.example.com/runbooks/service-health"
  severity: P2

flow:
  - step:
      id: check_dns
      type: tool
      title: "Resolve DNS: {{ .hostname }}"
      tool:
        name: nslookup
        action: lookup
```

```
    args:
      host: "{{ .hostname }}"
  capture:
    dns_result: stdout
  contract:
    effects: ["network.probe"]
    reads: ["hostname"]
    writes: ["dns_result"]

- step:
  id: check_http
  type: tool
  title: "HTTP health check: {{ .hostname }}"
  tool:
    name: curl
    action: head
    args:
      url: "https://{{ .hostname }}/healthz"
  capture:
    http_result: stdout
  timeout: "{{ .default_timeout }}"

- step:
  id: evaluate
  type: branch
  title: Evaluate health results
  branches:
    - condition: '{{ contains .http_result "200" }}'
      label: Healthy
      steps:
        - step:
            id: resolved
            type: end
            title: Service is healthy
            outcome:
              category: resolved
```

```

        code: http_ok
- condition: '{{ not (contains .http_result "200") }}'
  label: Unhealthy
  steps:
    - step:
      id: escalate
      type: end
      title: Escalate to on-call
      outcome:
        category: escalated
        code: http_fail

```

4.3 Input and Output Declarations

4.3.1 Input Declarations

Inputs declare variables that must be resolved before execution begins. In v2 the `inputs` map is a top-level field (promoted from `meta.inputs`).

```

inputs:
  <name>:
    type: string | number | boolean | secret
    required: true | false           # default: true
    description: "..."
    default: "..."                 # valid only when required: false
    example: "..."
    pattern: "[a-z]+"                # regex validation (string only)
    from: <binding>                  # resolution source

```

type field The `type` field (new in v2) declares the value's type. Supported values:

- **string** — Default. UTF-8 text.
- **number** — Integer or float, passed as string in templates.
- **boolean** — `true` or `false`.
- **secret** — String, redacted in traces and log output.

from: binding syntax The **from** field specifies how the value is resolved. v2 binding forms:

Binding	Behavior
<code>prompt</code>	Ask the operator interactively at runtime
<code>env.<NAME></code>	Read from environment variable NAME
<code>file.<path></code>	Read first line from a local file
<code><provider>.<field></code>	Delegate to a registered input provider (e.g. <code>incident.id</code>)
<code>var.<name></code>	Copy from another variable or capture

If **from** is omitted, the field must have a **default** value or the runtime will reject the runbook at semantic validation time.

4.3.2 Output Declarations

Outputs declare values that a composable runbook exposes to its caller via the **invoke** step's output extraction.

outputs:

```
<name>:
  type: string | number | boolean | secret
  description: "... "
  from: captures.<captureName>    # must reference a capture variable
```

The **from: captures.*** binding is the only supported form in v2. The named capture must be written by at least one step in the runbook; the semantic validator checks this.

4.3.3 Expression Language

v2 retains **Go templates** (`text/template`) as the expression language for all interpolated fields (`title`, `args`, `when`, `condition`, `instructions`, etc.). The decision to retain Go templates is grounded in two facts:

1. The v1 template engine is battle-tested and broadly used in v1 runbooks.
2. Go templates' pipelines and custom function maps provide sufficient expressiveness for operational conditionals.

The template data context is the *run variable map*: all `vars`, resolved `inputs`, and accumulated `captures` merged into a single flat `map[string]string`. Access by dot notation: `{{ .hostname }}`.

Built-in template functions (v2 additions) v2 adds a standard function library accessible in all templates:

Function	Signature
<code>contains</code>	<code>(s, substr string) bool</code>
<code>hasPrefix</code>	<code>(s, prefix string) bool</code>
<code>hasSuffix</code>	<code>(s, suffix string) bool</code>
<code>trim</code>	<code>(s string) string</code>
<code>toLower</code>	<code>(s string) string</code>
<code>toUpper</code>	<code>(s string) string</code>
<code>split</code>	<code>(s, sep string) []string</code>
<code>join</code>	<code>(elems []string, sep string) string</code>
<code>env</code>	<code>(name string) string</code>
<code>now</code>	<code>() string (RFC3339)</code>
<code>runID</code>	<code>() string</code>
<code>not</code>	<code>(v bool) bool</code>
<code>eq, ne, lt, gt, le, ge</code>	Standard comparison

All functions from `text/template` (built-ins: `and`, `or`, `index`, `len`, `printf`, etc.) remain available.

Template errors In v2, a template evaluation error in a `condition` or `when` field is a hard step failure (not a soft skip). The step transitions to `failed` and execution halts unless `continue_on_fail: true` is set on the step.

4.4 Step Types (v2 Inventory)

Each step in flow is a `TreeNode`: either a `step` object (with optional `branches` and `iterate` siblings) or a standalone `iterate` block.

4.4.1 Common Step Fields

The following fields are available on every step type:

Field	Type	Required	Description
<code>id</code>	string	Yes	Unique within the runbook
<code>type</code>	enum	Yes	Step type (see below)
<code>title</code>	string	No	Template; displayed in TUI/trace
<code>subtitle</code>	string	No	Secondary template annotation
<code>when</code>	string	No	Guard expression; skips step if false
<code>timeout</code>	string	No	Duration (e.g. 30s); overrides default
<code>delay</code>	string	No	Pre-execution pause (e.g. 5s)
<code>retry</code>	object	No	Retry config (§4.4.1)
<code>continue_on_fail</code>	bool	No	Continue execution on failure
<code>scope</code>	string	No	Named scope for variable isolation
<code>export</code>	string array	No	Variables to export from scope
<code>capture</code>	map	No	Output capture bindings
<code>contract</code>	object	No	Behavioral contract for governance

Retry configuration

```

retry:
  max: 3          # maximum retry attempts (required)
  interval: "5s"  # initial wait between attempts
  backoff: linear  # linear | exponential (default: linear)

```

Step contract The `contract` field annotates a step with its behavioral properties for governance policy evaluation:

```

contract:
  effects:
    - network.probe
    - service.restart
  reads:  [hostname, port]

```

```
writes: [http_result]
deterministic: true
idempotent: false
secrets: [auth_token]
```

Effects are free-form strings matched against `governance.rules[].effects`. The `secrets` list names variables whose values must be redacted in traces.

4.4.2 Step Type: cli

Executes a local command directly. In the v2 runtime `cli` is the canonical command step; the v1 `with.argv` form is accepted in compatibility mode and rewritten to `cli` by the migration tool.

Field	Type	Required	Description
<code>command</code>	string	Yes	Executable name or full path
<code>args</code>	string array	No	Arguments (templates expanded)
<code>env</code>	map	No	Additional environment variables
<code>workdir</code>	string	No	Working directory
<code>stdin</code>	string	No	Template string piped to stdin
<code>capture</code>	map	No	<code>stdout</code> <code>stderr</code> <code>exit_code</code> → var

Governance checks against the `allowed_commands` / `denied_commands` lists and any matching `governance.rules[].effects` are evaluated before the command executes. If the rule action is `require-approval`, the step pauses for human approval before proceeding.

```
- step:
  id: tail_logs
  type: cli
  title: "Tail last 100 lines from {{ .log_file }}"
  command: tail
  args: ["-n", "100", "{{ .log_file }}"]
  capture:
    output: stdout
```

```

    exit: exit_code
  contract:
    effects: [filesystem.read]
    reads: [log_file]
    writes: [output]

```

4.4.3 Step Type: manual

Prompts a human operator for evidence, approvals, or free-form input. Renamed from `collector` in v1. A `manual` step does not execute any command; it pauses execution and waits for the operator to provide the declared evidence items.

Field	Type	Required	Description
<code>instructions</code>	string	No	Markdown prompt displayed to operator
<code>required_evidence</code>	array	No	Evidence items (§4.4.3)
<code>approvals</code>	object	No	Approval gate config
<code>choices</code>	object	No	Radio-button choice selection

Evidence items

```

required_evidence:
- kind: text
  name: summary_notes
- kind: checklist
  name: preflight
  items:
    - "Verified monitoring dashboard"
    - "Confirmed no other incidents in flight"
- kind: attachment
  name: screenshot

```

Approval gate

```

approvals:
  min: 2
  roles: [DRI, change-manager]

```

```
timeout: 4h
on_timeout: escalate          # escalate | fail | skip
escalate_to: [manager]
```

The `timeout` and `on_timeout` fields are new in v2, addressing the SLA enforcement gap identified in the research brief.

```
- step:
  id: validate_dashboard
  type: manual
  title: Validate monitoring dashboard
  instructions: |
    Check the Grafana dashboard for **{{ .hostname }}**.
    Confirm error rate is below 1% and P99 < 500 ms.
  required_evidence:
    - kind: checklist
      name: dashboard_check
      items:
        - "Error rate < 1%"
        - "P99 latency < 500ms"
  approvals:
    min: 1
    roles: [DRI]
    timeout: 30m
    on_timeout: fail
```

4.4.4 Step Type: tool

Invokes a named tool action. The tool must be declared in `toolRefs`. The runtime dispatches to the tool's configured transport (stdio, jsonrpc, or mcp).

Field	Type	Required	Description
<code>tool.name</code>	string	Yes	Tool name (must match tool-Refs entry)
<code>tool.action</code>	string	Yes	Action declared in tool definition
<code>tool.args</code>	map	No	Argument key-value map (templates expanded)
<code>tool.version</code>	string	No	Optional version pin
<code>capture</code>	map	No	<code>stdout stderr json.<path> → var</code>

Output capture in v2 adds a `json.<path>` form for JSON-structured tool outputs, eliminating the need for follow-up `cli` steps with `jq`:

```
capture:
  pod_name: json.items[0].metadata.name
  status:   json.items[0].status.phase

- step:
  id: list_pods
  type: tool
  title: "List pods in {{ .namespace }}"
  tool:
    name: kubectl
    action: get-pods
    args:
      namespace: "{{ .namespace }}"
      output: json
  capture:
    pod_count: json.items | length
    first_pod: json.items[0].metadata.name
  contract:
    effects: [k8s.read]
    reads: [namespace]
    writes: [pod_count, first_pod]
```

4.4.5 Step Type: invoke

Invokes another runbook inline as a sub-procedure. The invoked runbook's outputs are mapped into the caller's variable space.

Field	Type	Required	Description
<code>invoke.runbook</code>	string	Yes	Import alias or relative path
<code>invoke.inputs</code>	map	No	Input bindings (template-expanded values)
<code>invoke.outputs</code>	map	No	Output extraction: local-var → output-name
<code>invoke.gate</code>	object	No	Stop-if conditions on child outcome

```
imports:
  connectivity: ./connectivity-check.runbook.yaml

flow:
  - step:
      id: run_connectivity
      type: invoke
      title: Run connectivity check
      invoke:
        runbook: connectivity
        inputs:
          service_name: "{{ .hostname }}"
          incident_id:  "{{ .incident_id }}"
        outputs:
          conn_status: health_status    # local var ← child output
      gate:
        stop_if: [escalated]
```

4.4.6 Step Type: branch

Evaluates a list of conditions and executes the first matching arm. In v2, **branch** is an explicit first-class step type (as well as the inline **branches:** sibling on any step node).

Field	Type	Required	Description
<code>branches</code>	array	Yes	List of Branch objects
<code>branches[].condition</code>	string	Yes	Go template evaluating to <code>true/false</code>
<code>branches[].label</code>	string	No	Display label for diagram/trace
<code>branches[].steps</code>	array	No	Steps to execute if condition matches

At most one branch arm executes. If no arm matches and no arm has an `else: true` marker, execution continues past the branch node with no steps run. The `else: true` marker (new in v2) designates a catch-all arm:

```
- step:
  id: route_on_env
  type: branch
  title: Route by environment
  branches:
    - condition: '{{ eq .environment "production" }}'
      label: Production path
      steps:
        - step:
            id: require_approval
            type: manual
            title: Require change-manager approval
            approvals:
              min: 1
              roles: [change-manager]
    - condition: '{{ eq .environment "staging" }}'
      label: Staging path
      steps:
        - step:
            id: log_staging
            type: cli
            title: Log staging deploy
            command: echo
            args: ["Deploying to staging"]
```

```

- else: true
  label: Unknown environment - abort
  steps:
    - step:
      id: abort
      type: end
      title: Unknown environment
      outcome:
        category: escalated
        code: unknown_env

```

4.4.7 Step Type: iterate

Repeats a block of steps over a collection or until a convergence condition is met. In v2 `iterate` is a `TreeNode` sibling (not a step type in the `step.type` enum) — it appears at the tree level alongside `step` nodes.

Field	Type	Required	Description
<code>iterate.over</code>	string	*	Template resolving to comma-separated list
<code>iterate.as</code>	string	No	Loop variable name (default: <code>item</code>)
<code>iterate.max</code>	int	*	Max iterations (convergence mode)
<code>iterate.until</code>	string	*	Convergence condition expression
<code>iterate.collect</code>	map	No	Accumulate captures across passes
<code>iterate.steps</code>	array	Yes	Steps to execute each pass

* `over` or (`max` + `until`) required; mutually exclusive.

The loop variable `{{ .iteration }}` (0-indexed) is always available. In list mode, `{{ .item }}` (or the `as` name) holds the current element.

```

# List iteration
- iterate:
  id: check_all_services

```

```
over: "{{ .service_list }}" # e.g. "api,worker,scheduler"
as: svc
collect:
  results: "{{ .svc }}:{{ .http_result }}"
steps:
  - step:
      id: probe
      type: tool
      title: "Check {{ .svc }}"
      tool:
        name: curl
        action: head
        args:
          url: "https://{{ .svc }}.internal/healthz"
      capture:
        http_result: stdout

# Convergence iteration
- iterate:
  id: wait_for_deploy
  max: 10
  until: '{{ eq .deploy_status "Running" }}'
  steps:
    - step:
        id: poll_deploy
        type: tool
        title: "Poll deployment (pass {{ .iteration }})"
        tool:
          name: kubectl
          action: get-deploy-status
          args:
            name: "{{ .deployment }}"
        capture:
          deploy_status: json.status.phase
    - step:
        id: wait
```

```

type: cli
title: Wait 15s
command: sleep
args: ["15"]

```

4.4.8 Step Type: parallel

Executes a set of independent step groups concurrently, then joins. New in v2, addressing the fan-out/fan-in gap identified in Dennis's research brief.

Field	Type	Required	Description
branches	array	Yes	Each branch is an independent group
join.wait_for	enum	No	all any majority (default: all)
join.on_failure	enum	No	fail continue (default: fail)

Parallel branches are isolated from each other's variable writes during execution. After the join, all **capture** values from all branches are merged into the caller's variable map. Write conflicts (two branches writing the same variable) are resolved by last-writer-wins and a warning is emitted.

```

- step:
  id: parallel_checks
  type: parallel
  title: Run DNS and HTTP checks concurrently
  branches:
    - label: DNS check
      steps:
        - step:
          id: dns_check
          type: tool
          tool:
            name: nslookup
            action: lookup
            args:

```

```
        host: "{{ .hostname }}"
      capture:
        dns_result: stdout
- label: HTTP check
  steps:
    - step:
      id: http_check
      type: tool
      tool:
        name: curl
        action: head
        args:
          url: "https://{{ .hostname }}/healthz"
      capture:
        http_result: stdout
  join:
    wait_for: all
    on_failure: fail
```

4.4.9 Step Type: assert

Evaluates a list of assertions against captured variables. Fails the run if any assertion does not pass.

```
- step:
  id: verify_deployment
  type: assert
  title: Verify deployment succeeded
  assert:
    - type: contains
      subject: "{{ .deploy_output }}"
      expected: "successfully rolled out"
    - type: eq
      subject: "{{ .exit_code }}"
      expected: "0"
    - type: json_path
      subject: "{{ .api_response }}"
```

```
path: "$.status"
expected: "active"
```

Assertion types: `contains`, `not_contains`, `matches` (regex), `eq`, `ne`, `lt`, `gt`, `exit_code`, `json_path`.

4.4.10 Step Type: `compensate`

New in v2. Registers a compensation (rollback) action that is executed in reverse order if the run fails or is cancelled after this step. Implements the saga pattern for long-running operational transactions.

Field	Type	Required	Description
<code>compensate.on</code>	enum	No	<code>failure</code> <code>cancel</code> <code>any</code> (default: <code>failure</code>)
<code>compensate.steps</code>	array	Yes	Steps to execute as compensation

The compensation steps execute in LIFO order relative to the sequence of `compensate` nodes encountered during the forward execution pass. Compensation steps share the same variable context as the point at which the `compensate` node was registered.

```
- step:
  id: scale_down
  type: tool
  title: Scale down service
  tool:
    name: kubectl
    action: scale
    args:
      deployment: "{{ .deployment }}"
      replicas: "0"

- step:
  id: register_scale_up
  type: compensate
  title: Compensation - scale back up on failure
```

```

compensate:
  on: failure
  steps:
    - step:
        id: scale_up_comp
        type: tool
        title: Restore service scale
        tool:
          name: kubectl
          action: scale
          args:
            deployment: "{{ .deployment }}"
            replicas: "{{ .original_replicas }}"

```

4.4.11 Step Type: end

Declares a terminal outcome and halts execution. Renamed from `router` in v1.

```

- step:
  id: resolved
  type: end
  title: Incident resolved
  outcome:
    category: resolved      # resolved | escalated | no_action | needs_rca
    code: dns_healthy
  meta:
    resolution_time: "{{ .elapsed }}"

```

4.5 Tool Definition Schema (v2)

Tool definitions describe callable tools: their actions, argument schemas, transport, and behavioral contract. In v2 the `tool/v0` version is replaced by `tool/v2`.

4.5.1 Tool Definition Field Inventory

```
$schema: "https://schemas.gert.dev/tool/v2.json"
```

```
apiVersion: tool/v2

meta:
  name: kubectl
  version: "1.2"
  description: Kubernetes CLI wrapper
  binary: kubectl

transport:
  mode: stdio          # stdio | jsonrpc | mcp
  # jsonrpc options:
  # start: ["node", "server.js"]
  # port: 3000
  # mcp options:
  # start: ["python", "mcp_server.py"]

actions:
  get-pods:
    description: List pods in a namespace
    argv: ["get", "pods", "-n", "{{ .namespace }}", "-o", "json"]
    args:
      namespace:
        type: string
        required: true
        description: Kubernetes namespace
    output:
      type: string
      required: false
      default: json
    capture:
      stdout:
        format: json    # json | text | lines
    contract:
      effects: [k8s.read]
      reads: [namespace]
    governance:
```

```
    action: allow
```

New in v2

- `capture.stdout.format` — declares the output format; enables structured `json.<path>` capture in runbook steps.
- `actions.<name>.contract` — per-action behavioral contract automatically merged with any step-level contract override.
- `actions.<name>.governance` — per-action governance policy; overrides the runbook-level rules for this action only.
- `meta.version` — explicit tool version for auditing and pinning.

4.6 Provider Definition Schema (v2)

Provider definitions declare external input resolution services. In v2 the `provider/v0` version is replaced by `provider/v2`.

```
$schema: "https://schemas.gert.dev/provider/v2.json"
apiVersion: provider/v2
```

```
meta:
  name: incident
  version: "1.0"
  description: PagerDuty incident context provider
  kind: input-provider # input-provider | enrichment-provider
```

```
transport:
  mode: jsonrpc
  start: ["/providers/pagerduty-provider"]
```

```
fields:
  id:
    type: string
    description: Active incident ID
```

```
title:
  type: string
  description: Incident title
severity:
  type: string
  enum: [P1, P2, P3, P4]
affected_service:
  type: string
  description: Primary impacted service name

capabilities:
  - input-resolution
```

The `fields` map defines the provider's schema. Each field name corresponds to a `from: <provider>.<field>` binding in a runbook input. The semantic validator checks that all `from: <provider>.*` bindings reference a field declared in a loaded provider definition.

4.7 Namespace Convention for Extensions

4.7.1 The Extension Field Problem

Strict JSON Schema validation (`additionalProperties: false`) is a core v2 design constraint: unknown fields in a runbook must be rejected, not silently ignored. At the same time, extensions must be able to attach metadata to steps, runbooks, and tool definitions without forking the core schema.

4.7.2 Convention: `x-namespace`: at the Field Level

v2 adopts the `x-namespace` prefix convention used by OpenAPI and GitHub Actions. Extension fields must:

1. Start with `x-`.
2. Be followed by a namespace component (lowercase alphanumeric and hyphens, matching `[a-z][a-z0-9-]+`).
3. Be declared in an **extension manifest** registered via `gert extension register` or in `gert.yaml`.

At the runbook level

```
$schema: "https://schemas.gert.dev/runbook/v2.json"
apiVersion: runbook/v2
id: deploy-service
name: Deploy Service

x-myorg-cost-center: "engineering-ops"
x-myorg-change-id: "CHG-20260418-001"
x-otel-service-name: "payment-service"
```

At the step level

```
- step:
  id: deploy
  type: tool
  title: Deploy to production
  x-myorg-risk-level: high
  x-myorg-rollback-id: "{{ .deploy_id }}"
```

4.7.3 Extension Validation

1. **Unregistered extension fields** — The JSON Schema validator accepts any field matching `^x-[a-z][a-z0-9-]+$` without requiring a registered schema. This enables zero-friction annotation. The `gert validate` command emits a `WARNING` for each unregistered extension field.
2. **Registered extension fields** — An extension may ship a JSON Schema fragment that constrains its field values. Once registered, the validator applies the fragment and reports violations as errors, not warnings.
3. **Unknown non-extension fields** — Any field that does not match a core field name and does not start with `x-` is a hard validation error (`additionalProperties: false`).

4.7.4 Extension Registration Format

```
# .gert/extensions/myorg.yaml
```

```
apiVersion: extension/v1
namespace: myorg
description: MyOrg internal runbook annotations

fields:
  cost-center:
    type: string
    description: Cost center code for billing
    pattern: "[a-z]+-[a-z]+"
  risk-level:
    type: string
    enum: [low, medium, high, critical]
  change-id:
    type: string
    pattern: "^CHG-[0-9]{8}-[0-9]{3}$"
  rollback-id:
    type: string

applies-to: [runbook, step]
```

The extension schema is merged into the live JSON Schema when the extension is registered, enabling IDE auto-completion for extension fields as well.

4.8 Structural vs. Semantic Validation

v2 validation runs in two sequential phases. Both must pass for a runbook to be considered valid.

4.8.1 Phase 1: Structural Validation

Structural validation is pure JSON Schema Draft 2020-12 evaluation. It checks:

- **Required fields** present (id, name, apiVersion, flow).
- **Field types** correct (string vs. integer vs. object vs. array).
- **Enum values** valid (e.g. kind must be one of the four defined values).

- **Pattern constraints** satisfied (e.g. `id` must match slug pattern; `timeout` must match `[0-9]+(s|m|h)`).
- **minItems** / **minLength** constraints met (e.g. `flow` must have at least one node; `choices.options` must have at least two options).
- **additionalProperties** — no unknown core fields present.

Structural validation is performed by the generated JSON Schema artifact (see §4.9). It is stateless: it does not load tool definitions, resolve imports, or evaluate templates.

Implementation note for Brian The Go runtime uses the `invopop/jsonschema` library to generate the JSON Schema from struct tags at startup (no static file required), then validates the parsed YAML document against it. The `yaml.v3 KnownFields(true)` call provides an additional pre-JSON-Schema pass that rejects unknown fields before the struct is even unmarshalled.

4.8.2 Phase 2: Semantic Validation

Semantic validation enforces domain rules that cannot be expressed in JSON Schema. It runs after structural validation and requires the full runbook AST and loaded tool/provider definitions. Semantic checks include:

1. **Step ID uniqueness** — All step IDs in the flat walk of the tree must be unique within the runbook.
2. **Invoke target resolution** — Every `invoke.runbook` value must resolve to a declared `imports` alias or a relative file path that exists on disk.
3. **Branch condition variable references** — Each `condition` template references only variables that are in scope at that point in the execution tree (declared as `inputs`, `vars`, or captured by an earlier step on a reachable path).
4. **Tool reference resolution** — Every `tool.name` in a tool step must match a declared `toolRefs` entry, and the named `.tool.yaml` file must be loadable and structurally valid.
5. **Tool action existence** — The `tool.action` must be declared in the referenced tool definition.

6. **Provider binding resolution** — Every `inputs.*.from: <provider>.<field>` must resolve to a loaded provider whose field manifest declares that field name.
7. **Circular invoke detection** — The invoke graph must be a DAG; any cycle is a semantic error.
8. **Capture write-before-read** — A step that reads a captured variable via a template must have a reachable predecessor step that writes it; a warning is emitted if the write is only on some branches.
9. **Governance policy pre-check** — The static governance rules are evaluated against the step contracts declared in the runbook. Any step with an effect that would trigger **deny** is flagged as a semantic error. Steps that would trigger **require-approval** are flagged as warnings.
10. **Output declaration completeness** — Every declared `output` must reference a capture name that is written by at least one step.
11. **Compensate ordering** — Each `compensate` step must appear after the step it is compensating for in tree walk order.

Error reporting Both validation phases produce structured error reports with:

- Error code (e.g. SCH-001, SEM-007)
- File path and YAML line/column (where available)
- Human-readable message
- Suggested fix (where computable)

```
$ gert validate service-health.runbook.yaml
[SEM-003] step "check_http": tool action "head" not declared in
          tools/curl.tool.yaml (declared actions: get, post, download)
          --> service-health.runbook.yaml:34:7
[SEM-007] step "eval": template references variable "dns_result" which
          is only written on branch "DNS check" - may be unbound
          --> service-health.runbook.yaml:51:12
          hint: add a default value in vars: or guard with {{ if .dns_result }}
```

4.9 Schema Artifacts

4.9.1 Shipped Artifacts

gert v2 ships the following schema artifacts:

Artifact	Description
<code>runbook.v2.schema.json</code>	JSON Schema Draft 2020-12 for runbook documents
<code>tool.v2.schema.json</code>	JSON Schema for tool definitions
<code>provider.v2.schema.json</code>	JSON Schema for provider definitions

All three schemas are generated at runtime from Go struct tags via `invopop/jsonschema` and served by `gert schema export`. They are also embedded in the binary and served at `https://schemas.gert.dev/*` for IDE tooling.

4.9.2 gert schema export

Export runbook schema (default)

```
gert schema export
```

Export specific schema

```
gert schema export --kind runbook # runbook | tool | provider | bundle
```

Export full bundle (all three)

```
gert schema export --kind bundle > gert-v2-schemas.json
```

Validate a document against the exported schema

```
gert schema export | check-jsonschema --schemafile - runbook.yaml
```

The bundle format is a JSON object with three keys: `runbook`, `tool`, and `provider`.

4.9.3 VS Code Integration

The gert VS Code extension configures schema-backed YAML validation via the `yaml-language-server` extension:

```
# .vscode/settings.json (auto-generated by gert vscode init)
{
```

```
"yaml.schemas": {  
  "https://schemas.gert.dev/runbook/v2.json": "**/*.runbook.yaml",  
  "https://schemas.gert.dev/tool/v2.json":    "**/*.tool.yaml",  
  "https://schemas.gert.dev/provider/v2.json": "**/*.provider.yaml"  
}
```

Because the schema URL matches the `$schema` field embedded in each document, IDEs with built-in JSON Schema support (VS Code 1.80+, IntelliJ 2024+) pick up validation and auto-completion without any per-project configuration.

The gert VS Code extension additionally provides:

- Step-type-aware field completion via the `schema/stepFields` JSON-RPC method (returns the field set for a given `type` value).
- Tool argument completion via `schema/toolArgs` (returns the `args` schema for a given tool+action pair from the live tool catalog).
- Inline governance warnings derived from `exec/dryRun` pre-check.

4.9.4 Schema Version Manifest

Each schema artifact carries a version manifest in its `$id` and a `x-gert-version` annotation:

```
{  
  "$schema": "https://json-schema.org/draft/2020-12/schema",  
  "$id": "https://schemas.gert.dev/runbook/v2.json",  
  "x-gert-version": "2.0.0",  
  "x-gert-released": "2026-04-18",  
  "title": "gert runbook v2",  
  ...  
}
```

Automated compatibility testing (part of the gert CI pipeline) runs every v1 example runbook through the v2 compatibility parser and asserts no regression, and runs every v2 example through the exported JSON Schema validator.

Chapter 5

Extension Runtime

The gert v2 extension runtime defines how external packages contribute tools, schema fields, governance policies, and input providers to a running gert host without requiring changes to the host binary. Extensions run as child processes; they communicate with the host over JSON-RPC 2.0 and are governed by an explicit capability grant model.

A normative reference specification is maintained at `specs/002-extension-runtime-v0/spec.md` in the repository. This chapter derives from and supersedes that document for design purposes; in any conflict between the two, the specification takes precedence for implementation.

5.1 Architecture Overview

1. The gert host discovers extension manifests on startup.
2. The host evaluates compatibility and capability policy before launching any process.
3. Each accepted extension is started as a child process; its executable communicates with the host over its declared transport.
4. The host and extension perform a versioned handshake. The host sends the set of capabilities it is willing to grant; the extension confirms receipt.
5. The extension registers its contributions (tools, schema fields, policies, providers).
6. For the remainder of the run the host dispatches invocations to the extension and enforces all capability constraints.
7. At run completion the host sends a graceful shutdown signal.

5.2 Capability Taxonomy

Every operation an extension may perform is gated by a named capability. Capabilities are requested in the manifest and granted (or denied) by host policy before the process is started. The initial capability set is given in Table 5.1.

The capability string format is `capability/<name>`. Future versions may introduce scoped variants using path notation, e.g. `capability/file-read:logs/**`.

5.3 Extension Manifest Format

Each extension ships a `gert-extension.yaml` file alongside its executable. The manifest declares identity, requested capabilities, transport, and compatibility constraints.

```
# gert-extension.yaml - Extension Manifest
apiVersion: extension/v2

meta:
  name: acme.incident-pack          # Reverse-DNS namespaced identifier
  version: "1.2.0"                  # Semver
  description: "Acme incident resolution tools and providers"
  author: "Acme Platform Team <platform@acme.example>"

compatibility:
  api-version: ">=2.0.0 <3.0.0"    # Host API version range (semver)
  protocol-version: "2.0"          # Extension protocol version

entry-point:
  executable: "./bin/acme-extension"
  args: ["--config", "acme.yaml"]

transport: stdio-jsonrpc           # stdio-jsonrpc | grpc | mcp

capabilities:
  - capability/tool-registration
  - capability/provider-registration
  - capability/schema-extension
```

```
- capability/network
- capability/file-read

# Required when capability/file-read is declared:
file-read-paths:
  - "./runbooks/**"
  - "./data/**"

# Required when capability/network is declared:
network-hosts:
  - "api.pagerduty.com"
  - "*.acme.example"

# Required when capability/env-read is declared:
env-read-patterns:
  - "ACME_API_*"
  - "PD_TOKEN"

platform:                                # Optional OS/arch constraints
  os: ["linux", "darwin", "windows"]
  arch: ["amd64", "arm64"]
```

Manifest validation is performed by the host before the process is started. An extension with an unsatisfied `compatibility.api-version` range is rejected before any subprocess is created.

5.4 Extension Discovery

The host discovers extensions using the following resolution order, with later entries taking precedence:

1. **Built-in extensions:** compiled into the host binary; always available; no manifest required.
2. **Workspace config:** extensions declared in `.gert/extensions.yaml` at the workspace root.

3. **Runbook declaration:** extensions listed in the runbook's top-level `extensions:` field.
4. **CLI override:** `-extension <path>` flags provided at invocation time.

The `.gert/extensions.yaml` workspace config format:

```
# .gert/extensions.yaml
extensions:
  - path: "./extensions/acme-pack"    # directory containing gert-extension.yaml
  - path: "/opt/gert-extensions/audit-pack"
```

The runbook `extensions:` field:

```
apiVersion: runbook/v2
meta:
  name: incident-triage
extensions:
  - path: "./extensions/acme-pack"
```

When the same extension appears in multiple discovery sources, the host deduplicates by `meta.name`. If two distinct extensions share the same name, the one with the higher resolution precedence wins; the lower one is logged as a warning and skipped.

5.5 Handshake and Lifecycle Protocol

All messages use JSON-RPC 2.0. The host is always the initiator; the extension never sends unsolicited requests.

5.5.1 extension/initialize (host → extension)

Sent immediately after the extension process starts. The host communicates the capabilities it is willing to grant and its own version identity.

Request params:

```
{
  "jsonrpc": "2.0",
```

```
"id": "1",
"method": "extension/initialize",
"params": {
  "host": {
    "name": "gert",
    "version": "2.0.0",
    "apiVersion": "2.0"
  },
  "protocolVersion": "2.0",
  "grantedCapabilities": [
    "capability/tool-registration",
    "capability/network"
  ],
  "runContext": {
    "runId": "run-abc123",
    "mode": "real"
  }
}
}
```

Success response:

```
{
  "jsonrpc": "2.0",
  "id": "1",
  "result": {
    "extension": {
      "name": "acme.incident-pack",
      "version": "1.2.0"
    },
    "protocolVersion": "2.0",
    "acknowledgedCapabilities": [
      "capability/tool-registration",
      "capability/network"
    ]
  }
}
```

Error codes for this method:

- -32001: Protocol version not supported by extension.
- -32002: Required capability not granted (extension considers the capability non-negotiable).
- -32003: Host API version outside extension compatibility range.

5.5.2 contributions/list (host → extension)

Sent after a successful `extension/initialized` notification. The extension returns its contribution manifest, filtered to the granted capability set.

```
{
  "jsonrpc": "2.0",
  "id": "2",
  "method": "contributions/list",
  "params": {}
}
```

Response:

```
{
  "jsonrpc": "2.0",
  "id": "2",
  "result": {
    "tools": [ { "name": "acme.pd-lookup", ... } ],
    "providers": [ { "name": "pd", "prefixes": ["pd."] } ],
    "schemaExtensions": [],
    "policyContributions": []
  }
}
```

5.5.3 extension/ping (host → extension)

Periodic health check. The extension **MUST** respond within the host-configured ping timeout (default 5 seconds). A non-response triggers a crash-recovery cycle (Section 5.7.3).

```
{ "jsonrpc": "2.0", "id": "42", "method": "extension/ping", "params": {} }
```

Response:

```
{ "jsonrpc": "2.0", "id": "42", "result": { "status": "ok" } }
```

5.5.4 extension/shutdown (host → extension)

Graceful termination. The extension SHOULD drain in-flight requests and release resources within the shutdown timeout (default 10 seconds).

```
{ "jsonrpc": "2.0", "id": "99", "method": "extension/shutdown", "params": {} }
```

Response (best-effort; host may not wait):

```
{ "jsonrpc": "2.0", "id": "99", "result": { "status": "ok" } }
```

If the extension does not exit within the shutdown timeout, the host sends SIGTERM followed by SIGKILL after a grace period.

5.5.5 Lifecycle State Machine

1. **discovered**: manifest found and parsed; no process yet.
2. **verified**: manifest validated; capabilities evaluated against policy.
3. **starting**: executable launched; stdin/stdout pipes attached.
4. **initializing**: `extension/initialize` sent; awaiting response.
5. **ready**: `contributions/list` completed; accepting invocations.
6. **draining**: `extension/shutdown` sent; waiting for in-flight completions.
7. **stopped**: process has exited cleanly.
8. **crashed**: process exited unexpectedly (see Section 5.7.3).

5.6 Versioning and Compatibility

The host maintains a monotonically increasing API version string following semver. The extension manifest declares a semver range in `compatibility.api-version`. Compatibility is evaluated before any process is started:

- If the host API version falls outside the declared range, the extension is rejected with a structured error logged to the run trace. The run continues with the remaining available extensions; a warning is surfaced to the operator.
- During `extension/initialize`, the host also sends its `protocolVersion`. If the extension does not support that protocol version it MUST return error code `-32001`.
- If an extension requests a capability that the host version does not implement (e.g., a capability introduced in a later host version), the capability is silently absent from `grantedCapabilities`. The extension SHOULD treat absent non-required capabilities as optional degradation rather than a fatal error.

Host API versions are tracked in `specs/api-versions.md`. Each new capability addition constitutes a minor version bump; breaking changes to existing method contracts constitute a major version bump.

5.7 Sandboxing and Process Isolation

5.7.1 Process Isolation

Extensions always run as child processes of the gert host. They are never loaded as shared libraries or plug-ins executed in-process. The host:

- Sets a restricted environment: only env vars matching `env-read-patterns` from the manifest are forwarded.
- Attaches the extension's stderr to a structured log collector; raw stderr lines are stored in the run trace under `extension.<name>.stderr`.
- Does not forward the host process's file descriptor table beyond stdin/stdout/stderr.

5.7.2 Capability Enforcement

Every RPC method the extension may call is guarded by a capability check. The host maintains a granted-capability set per extension process. Before dispatching any extension-initiated request (e.g., a tool registration during `contributions/list`) the host verifies the required capability is in the granted set. Violations return:

```
{
  "jsonrpc": "2.0",
  "id": "...",
  "error": {
    "code": -32010,
    "message": "capability not granted",
    "data": {
      "required": "capability/file-write",
      "granted": ["capability/file-read", "capability/network"]
    }
  }
}
```

File-path and network-host constraints are enforced by the host at the system call boundary using OS-level mechanisms where available (e.g., `pledge/unveil` on OpenBSD, `seccomp` on Linux) and by host-side validation of all path and URL arguments on other platforms.

5.7.3 Timeout and Crash Handling

- **Per-invocation timeout:** each `tools/invoke` call carries a `timeoutMs` field. If the extension does not respond within that deadline, the host sends a `tools/cancel` request and, if still unresponsive after a grace period, terminates the process.
- **Ping timeout:** if an extension fails to respond to `extension/ping` within 5 seconds it is considered unresponsive. The host logs a structured warning and terminates the process.
- **Unexpected exit:** if the extension process exits with a non-zero status or is killed by a signal, the host marks the extension as **crashed**, emits a `extension.crashed` trace event, and cancels all in-flight invocations with a structured `-32099` (extension crashed) error.

- **Host isolation:** extension crashes never crash the host. A run that loses an extension mid-execution surfaces an error on the affected step and allows the operator to decide whether to abort or continue with a degraded tool set.

Capability name	Allows	Excludes
capability/tool-registration	Register new tool definitions at runtime via <code>contributions/list</code> .	Does not allow the extension to invoke other tools or modify existing tool definitions.
capability/schema-extension	Contribute namespaced additional fields to the runbook schema (e.g. <code>x-acme.*</code> annotations).	Cannot modify or remove existing schema fields; cannot alter core validation rules.
capability/event-subscription	Subscribe to host runtime events (step-started, step-completed, run-ended, etc.) via the <code>events/subscribe</code> method.	Cannot emit synthetic events into the host event bus.
capability/policy-contribution	Register additional governance policy rules evaluated during pre-flight and approval checks.	Cannot override or disable built-in governance rules; cannot grant capabilities to other extensions.
capability/provider-registration	Register new input providers that handle <code>from:</code> prefix bindings.	Cannot intercept resolution requests destined for another registered provider.
capability/file-read	Read files within the path scope declared in the manifest (<code>file-read-paths</code>).	Cannot read files outside declared paths; cannot write or delete files.
capability/file-write	Write or create files within the path scope declared in the manifest (<code>file-write-paths</code>).	Cannot write outside declared paths; does not imply read access.
capability/network	Make outbound network calls to the hosts declared in the manifest (<code>network-hosts</code>).	Cannot connect to undeclared hosts; cannot listen for inbound connections.
capability/env-read	Read environment variables whose names match the declared <code>env-read-patterns</code> list.	Cannot read undeclared variables; cannot write or unset environment variables.
capability/run-state-read	Read current run state (step status, captured variables, trace events).	Cannot modify run state; does not imply the ability to emit output.

TABLE 5.1: Extension capability taxonomy (v2).

Chapter 6

Tool Runtime

The tool runtime is the subsystem responsible for resolving, validating, and invoking tool definitions at runbook execution time. Tools are the primary mechanism by which gert performs side-effecting work: executing binaries, calling JSON-RPC services [10], and delegating to MCP-hosted capabilities [3].

6.1 Tool Discovery

The host resolves tool references in three phases:

1. **Static discovery:** `.tool.yaml` files are located by scanning the following directories in order:
 - (a) The built-in tool registry compiled into the host binary.
 - (b) `<workspace-root>/tools/` — the conventional location for project-level tool definitions (`tools/<name>.tool.yaml`).
 - (c) Directories listed in the workspace config (`.gert/config.yaml`, field `tool-paths`).
 - (d) Required packages declared in the runbook's `requires:` field (each package's `tools/` subdirectory is scanned).
2. **Dynamic registration:** extensions with `capability/tool-registration` may register additional tool definitions during the `contributions/list` phase (see Section 5.5). Dynamically registered tools are merged into the resolved catalog before execution begins.

3. **MCP discovery**: when the runbook declares an MCP server under `tools.mcp-servers`, the host performs a `tools/list` call to that server at startup and registers every tool reported as a dynamically discovered tool (see Section 6.7).

6.1.1 Name Resolution Order

When a runbook step references a tool by name, the host resolves as follows:

1. Exact match in dynamically registered tools (MCP and extension, in registration order).
2. Exact match in project-level `tools/` directory.
3. Exact match in required-package tool directories (in `requires:` declaration order).
4. Exact match in the built-in registry.

If two tools share the same name, the first match wins and a warning is emitted. Tool authors SHOULD use reverse-DNS namespace prefixes to avoid collisions (e.g., `acme.pd-lookup`).

6.2 Tool Definition Schema (v2)

```
# tools/my-service.tool.yaml
apiVersion: tool/v2

meta:
  name: my-service           # Unique identifier within the workspace
  version: "2.0.0"          # Tool definition version (semver)
  description: "Calls the MyService REST API"
  binary: my-service-cli     # Executable name (looked up in PATH)

transport:
  mode: stdio-jsonrpc        # stdio | stdio-jsonrpc | grpc | mcp

# For stdio-jsonrpc and mcp: startup behaviour
startup:
  argv: ["--server-mode"]    # Extra args passed to the binary
```

```
ready-pattern: "ready"      # Regex matched against stdout/stderr to
                             # confirm the server is accepting requests
timeout: "10s"
shutdown-method: "shutdown" # JSON-RPC method to call on graceful stop

governance:
  requires-capabilities: []      # Host capabilities the tool needs, e.g.
                                # ["capability/network"]
  allowed-environments: ["real", "dry-run"]
  requires-approval: false      # Set true to gate invocation on operator OK

actions:
  lookup:
    description: "Look up an entity by ID"
    argv: ["lookup", "--id", "{{ .id }}"] # Only for stdio transport
    timeout: "30s"
    args:
      id:
        type: string
        required: true
        description: "Entity identifier"
        redact: false
    output:
      format: json              # text | json | jsonl
    capture:
      stdout:
        format: json
```

6.2.1 Transport-Specific Fields

stdio The binary is spawned once per invocation. The `argv` array in the action definition is rendered as a Go template with the resolved argument values, then appended to the binary name to form the complete command. The process's stdout is captured as the tool output.

stdio-jsonrpc The binary is spawned once per run (persistent process). The host keeps a single process alive across all action calls for that tool. Each action invocation sends a JSON-RPC request to the process's stdin and reads the response from stdout. The `startup.ready-pattern` confirms the server is accepting connections.

mcp The host spawns the binary and performs the MCP initialization handshake (Section 6.7). Actions map to MCP `tools/call` requests.

6.3 Transport Layer

6.3.1 stdio Transport

Process establishment. The host calls `exec.Command(binary, resolvedArgv...)` with a controlled environment. A per-invocation timeout context wraps the call.

Invocation envelope. For stdio tools the invocation is entirely encoded in the process arguments and environment; there is no envelope. The arguments are rendered from the action's `argv` template.

Response envelope. Stdout is captured in full after process exit. The exit code is checked: non-zero is mapped to a tool error.

Error envelope.

```
{
  "error": {
    "code": "TOOL_NONZERO_EXIT",
    "message": "tool exited with code 1",
    "details": {
      "exitCode": 1,
      "stderr": "connection refused\n",
      "durationMs": 320
    }
  }
}
```

6.3.2 stdio-jsonrpc Transport

Process establishment. A single process is spawned at first use and kept alive for the duration of the run. The host waits for `startup.ready-pattern` on stdout/stderr before sending the first request.

Invocation envelope.

```
{
  "jsonrpc": "2.0",
  "id": "<invocationId>",
  "method": "<actionName>",
  "params": {
    "args": { "id": "INC-12345" },
    "context": {
      "runId": "run-abc123",
      "stepId": "lookup_incident",
      "traceId": "trace-xyz",
      "userId": "ops@acme.example",
      "attempt": 1
    }
  }
}
```

Response envelope.

```
{
  "jsonrpc": "2.0",
  "id": "<invocationId>",
  "result": {
    "output": { ... },
    "telemetry": {
      "startedAt": "2026-04-18T10:00:00Z",
      "finishedAt": "2026-04-18T10:00:00.320Z",
      "durationMs": 320
    }
  }
}
```

Error envelope.

```
{
  "jsonrpc": "2.0",
  "id": "<invocationId>",
  "error": {
    "code": -32050,
    "message": "entity not found",
    "data": {
      "category": "runtime",
      "retryable": false,
      "details": { "entityId": "INC-12345" }
    }
  }
}
```

Error code ranges:

- -32700 to -32600: JSON-RPC parse and request errors.
- -32050: Application-level tool error (retryable per `data.retryable`).
- -32051: Capability not granted.
- -32052: Input validation failed.
- -32053: Output schema validation failed.
- -32054: Timeout.
- -32055: Cancelled by host.
- -32099: Tool process crashed.

6.3.3 MCP Transport

See Section 6.7 for the full MCP integration specification.

6.4 Invocation Context

Every tool invocation — regardless of transport — receives a structured context object. For `stdio` tools, relevant fields are injected as environment variables prefixed `GERT_`; for `stdio-jsonrpc` and `mcp` tools, the context is transmitted in the invocation envelope.

```
{
  "context": {
    "runId":      "run-abc123",          // Globally unique run identifier
    "stepId":     "lookup_incident",    // Step ID within the runbook
    "traceId":    "trace-xyz789",       // Distributed trace correlation ID
    "userId":     "ops@acme.example",   // Identity executing the run
    "mode":       "real",               // real | dry-run | replay
    "attempt":    1,                   // Retry attempt number (1-based)
    "capabilities": [                  // Effective host capability set
      "capability/network",
      "capability/file-read"
    ]
  }
}
```

6.5 Capability Gate

Before dispatching any tool invocation the host checks two conditions:

1. The tool's `governance.requires-capabilities` list is a subset of the current run's granted capability set. If not, the step fails immediately with a `CAPABILITY_DENIED` error; the run does not attempt to start the tool process.
2. The tool's `governance.allowed-environments` list includes the current execution mode. A tool not listed for `dry-run` will be skipped (with a warning) when the run mode is `dry-run`.

6.6 Output Capture

Tool output is mapped to named step captures using the `capture:` declaration in the action definition.

```
capture:
  dns_output: stdout          # Capture stdout as variable dns_output
  error_log:  stderr          # Capture stderr as variable error_log
  exit_code:  exitCode        # Capture numeric exit code
```

For json output format, capture paths may use dot notation to extract nested fields:

```
capture:
  incident_id: "stdout.incident.id"
  severity:    "stdout.incident.severity"
```

Redaction. Before any captured value is stored in the run state or emitted to the trace, the governance redaction pipeline is applied. Redaction rules are declared in the runbook's `meta.governance.redact` block and in the tool action's per-argument `redact: true` flag. Redacted values are replaced with `<redacted>` in all stored and displayed output.

6.7 MCP Tool Integration

MCP (Model Context Protocol) servers are long-lived processes that expose a dynamic catalog of tools. Gert treats an MCP server as a tool source, not as a single tool.

6.7.1 MCP Server Declaration

```
# In runbook meta or .gert/config.yaml
tools:
  mcp-servers:
    - name: acme-mcp
      binary: acme-mcp-server
      args: ["--port", "stdio"]
      transport: mcp
      startup:
        timeout: "15s"
```

6.7.2 MCP Discovery

On run startup the host:

1. Spawns the MCP server binary.
2. Sends the MCP `initialize` request (per MCP specification).
3. After receiving `initialized`, sends `tools/list`.
4. Registers each returned tool under the name `<server-name>/<tool-name>` in the dynamic tool catalog.

6.7.3 MCP Tool Invocation

When a runbook step calls an MCP-backed tool, the host sends:

```
{
  "jsonrpc": "2.0",
  "id": "5",
  "method": "tools/call",
  "params": {
    "name": "lookup_incident",
    "arguments": { "id": "INC-12345" },
    "_gert_context": {
      "runId": "run-abc123",
      "stepId": "triage_step",
      "traceId": "trace-xyz"
    }
  }
}
```

The `_gert_context` field is a gert extension to the MCP envelope. MCP servers that do not recognise it **MUST** ignore it. The response follows the standard MCP `tools/call` response format.

6.7.4 MCP vs stdio-jsonrpc Differences

- MCP servers expose multiple tools under a single process; stdio-jsonrpc tools are one tool per process.
- MCP uses a distinct initialization handshake (`initialize` / `initialized`) before tool registration.

- MCP tool names are discovered dynamically at runtime; stdio-jsonrpc action names are declared statically in the `.tool.yaml`.
- MCP responses use a `content` array; gert flattens text content to a single string for capture.

6.8 Cancellation and Timeout

Per-tool timeout. Declared in the action definition under `timeout:` (e.g., 30s). The default is the runbook's `meta.defaults.timeout`. The host wraps each invocation in a `context.WithTimeout`; when the deadline elapses the host sends a `tools/cancel` request (JSON-RPC transport) or sends SIGTERM (stdio transport), then forcibly terminates the process after a 5-second grace period.

Cancellation propagation. When a run is cancelled (via `Ctrl-C`, the debugger quit command, or an API cancellation request), the host:

1. Sets the run-level cancel context.
2. Sends `tools/cancel` to all in-flight JSON-RPC and MCP invocations with a `reason: "run-cancelled"` field.
3. After a 5-second drain window, forcibly terminates any unresponsive tool processes.
4. Emits a `run.cancelled` trace event recording which steps were interrupted.

The `tools/cancel` request envelope:

```
{
  "jsonrpc": "2.0",
  "id": "cancel-5",
  "method": "tools/cancel",
  "params": {
    "invocationId": "5",
    "reason": "run-cancelled"
  }
}
```

6.9 Tool Versioning

Tool definitions carry a `meta.version` field. When multiple definitions for the same tool name are present (e.g., from a required package and from a workspace override), the host selects by resolution order (Section 6.1), not by version number. Explicit version pinning is supported via the runbook's `requires:` block:

```
requires:
- package: github.com/acme/gert-tools
  version: ">=1.2.0 <2.0.0"
```

Chapter 7

Runtime Events and Determinism

The runtime is event-first and deterministic.

7.1 Event Categories

- Planning events
- Step lifecycle events
- Branching and routing events
- Evidence and policy events
- Completion events

Adapters render from events and must not implement private execution logic.

Chapter 8

Security and Trust

8.1 Trust Model

- Extensions are untrusted by default.
- Capabilities are least-privilege and explicit.
- Policy controls extension admission and operation.

8.1.1 Runtime Hardening

- Host survives extension failure.
- Host enforces timeout, cancellation, and retry policy.
- Host logs structured diagnostics for extension interactions.

Chapter 9

Testing and Acceptance

This section specifies the complete testing strategy for gert v2: the test framework, unit and contract test patterns, integration test strategy, the `gert test` feature, regression testing against the v1 runbook corpus, performance acceptance criteria, the extension test harness, and CI/CD gate requirements.

9.1 Test Framework

9.1.1 Standard Library and testify

All gert v2 tests are written in Go using the standard `testing` package as the test runner and github.com/stretchr/testify [21] for assertions:

- `testify/assert` — non-fatal assertions; test continues on failure.
- `testify/require` — fatal assertions; test halts on failure.
- `testify/mock` — interface mocking for unit tests (engine, event bus, trace writer).

Table-driven tests are the preferred pattern for all logic that has well-defined input/output pairs (e.g. expression evaluation, schema validation, governance rule matching):

```
func TestGovernanceAllowlist(t *testing.T) {
    cases := []struct {
        name    string
        cmd     []string
        allowed bool
    }{
```

```
    {"exact match", []string{"kubectl", "get", "pods"}, true},
    {"partial match", []string{"kubectl", "delete", "ns"}, false},
    {"empty allowlist", []string{"echo", "hi"}, false},
  }
  for _, tc := range cases {
    t.Run(tc.name, func(t *testing.T) {
      result := allowlist.Check(tc.cmd)
      assert.Equal(t, tc.allowed, result)
    })
  }
}
```

9.1.2 Test Packages

Each Go package in the v2 module ships a co-located `_test.go` file (white-box tests) and optionally an `<pkg>_integration_test.go` file (integration tests, gated by a build tag). The naming convention:

```
pkg/engine/engine.go
pkg/engine/engine_test.go           // white-box unit tests
pkg/engine/engine_integration_test.go // build tag: //go:build integration
```

9.2 The gert test Feature

9.2.1 Scenario Replay Testing

`gert test <runbook.yaml>` discovers and executes all scenario tests for a runbook. A scenario test consists of:

- `inputs.yaml` — resolved input values for the run.
- `scenario.yaml` (optional) — pre-recorded command responses and manual evidence.
- `test.yaml` — assertions over the resulting trace and captures.

Scenario directories are located by convention:

```
{runbook-dir}/scenarios/{runbook-name}/{scenario-name}/
  inputs.yaml
  scenario.yaml  # optional; omit for runbooks with no CLI/tool steps
  test.yaml
```

The test runner executes the runbook in replay mode using the scenario file as the command/evidence fixture, then evaluates assertions from `test.yaml`.

9.2.2 Test Assertions (`test.yaml`)

```
assertions:
  outcome: passed          # "passed" | "failed" | "skipped"
  steps_passed: 5
  steps_failed: 0

captures:
  db_host:
    equals: "prod-db.example.com"
  migration_status:
    contains: "complete"

trace:
  - type: step/completed
    step_id: run-migration
  - type: governance/blocked
    step_id: drop-table
    data.rule: denylist
```

The `trace` assertions match events in order of occurrence. The `data.*` dotted paths dereference into the event's `data` object.

9.2.3 v1 Scenario Compatibility

v1 scenario files (`commands/evidence` structure) are forward-compatible with v2 without modification. The v2 parser accepts both the v1 format and the extended v2 format (which adds the `assertions` block). Runbook authors can migrate incrementally by adding `test.yaml` alongside existing v1 scenarios.

9.3 Contract Tests

Contract tests verify the interface boundaries between v2's five components. Each contract test lives in the *consumer* package and uses a conformance helper that validates any implementation satisfying the interface.

9.3.1 Core Runtime Contract

The runtime `Executor` interface must satisfy:

- Emits a `run/started` event before executing any step.
- Emits a `step/started` before and `step/completed/step/failed` after each step.
- Emits `run/completed` or `run/failed` as the final event.
- Does not emit events after the terminal event.
- Does not call any adapter method directly (events only).

```
// ConformanceTest validates any Runtime implementation.
func ConformanceTest(t *testing.T, rt Runtime) {
    t.Helper()
    bus := newTestEventBus()
    err := rt.Execute(context.Background(), fixtureRunbook, bus)
    require.NoError(t, err)
    events := bus.Events()
    require.Equal(t, "run/started", events[0].Type)
    require.Equal(t, "run/completed", events[len(events)-1].Type)
    assertNoEventsAfterTerminal(t, events)
}
```

9.3.2 Schema Contract Tests

- Every `apiVersion`: `runbook/v2` document in `testdata/` must pass structural validation without errors.
- Every invalid fixture in `testdata/invalid/` must fail validation with a structured error (message + path).

- Structural and semantic validation are tested independently: a structurally valid document may fail semantic validation (e.g. referencing an undefined capture).

9.3.3 Extension Manifest and Handshake Tests

The extension host verifies:

- An extension process that responds to the initialisation handshake within the configured timeout is accepted.
- An extension process that fails to respond is terminated and its capabilities are not registered.
- Capability grants in the manifest are enforced: an extension without `cap:schema.register` cannot register schema.
- A manifest with missing required fields is rejected at load time with a structured error.

9.3.4 Tool Invocation Envelope Tests

- Tool invocations carry the correct `run_id` and `step_id` correlation fields.
- Tool outputs are captured and available as step captures after the step completes.
- A tool that exits with non-zero triggers `step/failed` with `error_type: tool_error`.
- Timeout cancellation: a tool that exceeds its timeout receives a `context.DeadlineExceeded` and emits `step/failed` with `error_type: timeout`.

9.3.5 Event Bus Determinism Tests

- Events delivered to the event bus maintain the `seq` ordering guaranteed by `RunStore.WriteTrace`.
- Slow adapter consumers do not cause the engine's execution loop to block (bounded channel or dropping policy required).
- Event replay (`ReadTraceSince`) returns events in ascending `seq` order.

9.4 Integration Test Strategy

9.4.1 End-to-End Fixture Runs

Integration tests spin up the gert runtime against fixture runbooks stored in `testdata/runbooks/` and assert on the resulting trace files. They are gated by the `integration` build tag and run in CI separately from unit tests.

```
//go:build integration
```

```
func TestE2E_BranchingRunbook(t *testing.T) {
    dir := t.TempDir()
    runID, err := exec.RunFixture(dir, "testdata/runbooks/branch.runbook.yaml",
        WithInputs(map[string]string{"env": "prod"}),
        WithScenario("testdata/scenarios/branch-prod.yaml"),
    )
    require.NoError(t, err)
    trace := readTrace(t, dir, runID)
    assertEvent(t, trace, "step/completed", map[string]any{
        "data.step_id": "deploy-prod",
    })
    assertNoEvent(t, trace, "step/completed", map[string]any{
        "data.step_id": "deploy-staging",
    })
}
```

9.4.2 Fixture Runbook Corpus

The integration test corpus includes fixtures covering:

- Linear runbook (all step types: cli, manual, tool, invoke).
- Branching runbook (both branch arms; governance-blocked branch).
- Iterate runbook (list mode, convergence mode, parallel concurrency).
- Nested invoke (parent + two child runbooks).
- Retry policy (transient failure followed by success).

- Resumption (crash after step 2, resume, verify trace continuity).
- Governance: allowlist violation, denylist violation, approval gate.

9.4.3 Trace Assertion Helpers

A shared test package `pkg/testutil` provides helpers:

```
package testutil

// ReadTrace reads and parses all events from a run's trace.jsonl.
func ReadTrace(t *testing.T, runDir, runID string) []engine.DurableEvent

// AssertEvent asserts that at least one event matches type and data predicates.
func AssertEvent(t *testing.T, events []engine.DurableEvent,
    eventType string, predicates map[string]any)

// AssertEventOrder asserts events appear in the given type order.
func AssertEventOrder(t *testing.T, events []engine.DurableEvent,
    types ...string)

// AssertNoEvent asserts no event matches the given type and predicates.
func AssertNoEvent(t *testing.T, events []engine.DurableEvent,
    eventType string, predicates map[string]any)
```

9.5 Regression Testing

9.5.1 v1 Runbook Corpus as Fixtures

The v1 runbook corpus (located in `examples/` and operator-contributed runbooks) serves as a regression suite for v2 compatibility. The regression test runner:

1. Reads each v1 runbook from the corpus.
2. Runs `gert validate` (v2 schema) and asserts it passes (or passes after migration).
3. For runbooks with associated v1 scenario files, runs `gert test` and asserts the outcome matches the v1 golden output.

4. Fails the build if any previously passing corpus runbook regresses.

Migration compatibility is measured as: the percentage of v1 corpus runbooks that pass v2 `gert validate` without modification. The v2 launch target is $\geq 95\%$.

9.5.2 Golden Trace Comparisons

For deterministic scenarios, the expected trace can be stored as a golden file. The golden comparison test:

1. Runs the scenario.
2. Normalises timestamps and run IDs (replaces with fixed values).
3. Compares the normalised JSON-lines output to `testdata/golden/<scenario>.jsonl`.
4. Fails if any event differs. Golden files are updated by running with `-update` flag.

9.6 Performance Acceptance Criteria

The following latency budgets must pass on a reference machine (4-core, 8 GB RAM, SSD) with the integration test suite:

Operation	p50 target	p99 target
<code>gert validate</code> (single runbook, <50 steps)	<50 ms	<200 ms
Step startup (cli step)	<10 ms	<50 ms
Event delivery to adapter (local channel)	<1 ms	<5 ms
Trace event write + fsync	<5 ms	<20 ms
Snapshot write (100 steps history)	<20 ms	<80 ms
<code>gert test</code> (10-step replay scenario)	<500 ms	<1 s
Tool invocation round-trip (stdio, local process)	<50 ms	<200 ms

Performance benchmarks are written using Go's `testing.B` and run in CI weekly (not on every PR). A regression of $>2\times$ on any metric blocks the nightly build.

9.7 Extension Test Harness

9.7.1 Overview

Extension developers can test their extension logic without spinning up the full gert runtime. The `pkg/exttest` package provides a test harness:

```
import "github.com/ormasoftchile/gert/v2/pkg/exttest"

func TestMyExtension(t *testing.T) {
    h := exttest.NewHarness(t,
        exttest.WithManifest("testdata/my-ext.manifest.yaml"),
        exttest.WithCapabilities("cap:schema.register", "cap:tool.register"),
    )
    defer h.Close()

    // Start the extension process
    h.Start("./my-extension")

    // Assert it registered expected tools
    tools := h.RegisteredTools()
    require.Contains(t, tools, "my-tool")

    // Invoke a tool and assert the response
    resp := h.InvokeTool("my-tool", map[string]any{"arg1": "value"})
    assert.Equal(t, 0, resp.ExitCode)
    assert.Contains(t, resp.Stdout, "expected output")
}
```

9.7.2 Harness Capabilities

The test harness provides:

- A mock extension host that implements the full handshake protocol.
- Assertion helpers for registered tools, schema registrations, and event subscriptions.
- A `FakeRunContext` that provides a realistic run context without executing a runbook.

- Automatic process cleanup on test completion.
- Configurable capability grants (to test capability enforcement).

9.7.3 Mocking the Extension Transport

For pure unit tests of extension logic (without spawning a subprocess), the transport layer is mockable:

```
// InProcessTransport runs the extension handler in-process for unit testing.
type InProcessTransport struct {
    Handler ExtensionHandler
}
```

9.8 Acceptance Criteria

The following acceptance criteria must be satisfied before a v2 release is considered shippable:

1. Core runtime has no direct adapter dependencies — verified by the `no_import` contract test.
2. Extensions register schema and tools through declared contracts — verified by the extension handshake conformance suite.
3. Capability violations fail fast with structured errors — verified by the governance contract test for each capability type.
4. Adapters integrate using the same stable core contracts — verified by running each adapter’s integration test against a mock core.
5. Every step type (`cli`, `manual`, `tool`, `invoke`, `branch`, `iterate`) has ≥ 1 end-to-end integration test covering the happy path and at least one failure path.
6. The v1 runbook corpus passes `gert validate` at $\geq 95\%$ without modification.
7. All performance targets in § 8.6 pass on the reference machine.
8. `gert test` successfully replays all scenario fixtures in `testdata/scenarios/`.

9.9 CI/CD Integration

9.9.1 PR Gate (every pull request)

1. **Unit tests:** `go test ./...` (no build tags). Target: <60 s.
2. **Schema validation:** validate all YAML fixtures in `testdata/` against v2 JSON Schema. Target: <10 s.
3. **Contract tests:** `go test -run Contract ./...` Target: <30 s.
4. **Linters:** `golangci-lint` run with the project's `.golangci.yaml` config.
5. **Vet:** `go vet ./...`

All five gates must pass for a PR to be mergeable.

9.9.2 Integration Gate (merge to main)

1. **Integration tests:** `go test -tags=integration ./...` Target: <5 min.
2. **Regression corpus:** `gert test` against v1 corpus. Target: <2 min.
3. **Golden trace comparison:** normalised trace comparison for all deterministic fixtures.

9.9.3 Nightly Gate

1. **Performance benchmarks:** `go test -bench=. -benchtime=10s ./...` — compare against stored baseline; fail on >2× regression.
2. **Race detector:** `go test -race ./...`
3. **Extension conformance suite:** run all built-in extensions through the conformance harness.

Chapter 10

Open Questions

The questions below are unresolved architectural decisions. Each entry states **(a)** why the question matters for the design, **(b)** the options under consideration, and **(c)** what additional information is needed to close it. Decisions must be recorded in `.squad/decisions.md` before core contract freeze.

Q1. Can the v2 runtime execute v1 runbooks without prior migration?

Why it matters: Existing gert users have production runbooks in `runbook/v0` and `runbook/v1` format. If v2 can only execute v2-schema runbooks, adoption requires a migration step that may be blocked by organizational change control. A compatibility execution mode (“v1 shim”) removes this barrier but adds permanent maintenance cost to the parser.

Options: (a) Full compatibility mode: v2 runtime transparently executes v1 runbooks by internally upgrading them to v2 schema at load time, with no schema file written. (b) Migration gate: v2 refuses to run v1 runbooks and provides a clear error pointing to `gert migrate`. (c) Opt-in compatibility flag: `-compat-v1` enables the shim; production is v2-only.

Information needed: A survey of how many v1-format runbooks exist in representative deployments, and whether `gert migrate` can be made fully lossless for all v1 constructs.

Q2. What is the concurrency model for the core runtime?

Why it matters: The v1 runtime is single-run-per-process. Whether v2 supports multiple concurrent runs in a single process affects the state isolation model, the trace writer design, the governance enforcer (is it per-run or global?), and whether `gert serve` can multiplex concurrent VS Code sessions.

Options: (a) Single-run-per-process, matching v1 semantics; each `gert exec` or `gert tui` is a separate OS process. (b) Multiple concurrent runs in-process, with run-scoped state isolation via Go context and per-run goroutine groups. (c) Multiple runs via a long-lived daemon (`gert serve` as the host), with runs as logical sessions.

Information needed: Whether the VS Code extension use case requires concurrent runs in a single `gert serve` process (e.g., multiple open runbook panels), and what the goroutine safety requirements are for the trace writer and governance layer.

Q3. Is the v2 trace format backward-compatible with the v1 JSONL trace, or is it a new format?

Why it matters: Operators and compliance tools may have parsers, SIEM integrations, or dashboards built against the v1 JSONL event schema. A breaking change in the trace format requires those integrations to be updated before v2 can be rolled out. A backward-compatible superset format avoids this but constrains the v2 event schema design.

Options: (a) New format: v2 defines a clean JSONL schema with a required `schema_version` field; v1 trace parsers are unsupported. (b) Backward-compatible superset: v2 trace events include all v1 fields plus new fields; existing parsers continue to work on the subset they understand. (c) Parallel trace output: v2 writes both a v2 trace and an optional v1 compatibility trace (flag-enabled).

Information needed: An inventory of known downstream trace consumers and whether the v1 format has fields that would be structurally incompatible with the v2 event model.

Q4. How do extension-contributed policies compose with host-level governance policies?

Why it matters: The host enforces a governance policy (command allowlist, env blocking, redaction). An extension may also wish to contribute policies—e.g., a security extension that adds additional command denylists or redaction patterns. The composition model determines whether extensions can weaken host policies (prohibited), extend them (likely desired), or operate in a separate policy namespace (safest but most limited).

Options: (a) Additive composition only: extensions may only add restrictions (denylist entries, redaction patterns); they may never remove host-level restrictions. (b) Policy namespacing: each extension has an isolated policy scope; the host applies all active

policies via intersection. (c) Policy delegation: the host can explicitly delegate a policy domain to an extension (e.g., redaction rules owned by a data-classification extension).

Information needed: A threat model analysis of what happens when a malicious or compromised extension contributes a policy that attempts to weaken host restrictions. This should feed into §??.

Q5. Does the `.provider.yaml` input provider framework survive unchanged in v2?

Why it matters: v1's input provider model (`.provider.yaml`, JSON-RPC resolution, `from:` bindings, workspace config) is a complete and working framework. It must either be carried forward as-is, redesigned to fit the v2 component model, or replaced by an extension-based resolution mechanism. The choice affects the schema (§??), the extension runtime (§??), and any operator who has a custom provider binary.

Options: (a) Preserve as-is: `.provider.yaml` format and JSON-RPC protocol are unchanged; providers are a stable first-class concept. (b) Redesign as extension capability: providers become a named capability that extensions may implement; the `.provider.yaml` format is deprecated. (c) Hybrid: `.provider.yaml` providers continue to work as a built-in extension type, with a migration path to the full extension protocol.

Information needed: Whether any v1 provider binaries are in active production use that would be broken by a protocol change, and whether the extension handshake protocol can be made a strict superset of the provider JSON-RPC protocol.

Q6. Is the `gert serve` JSON-RPC contract a versioned breaking change or an evolutionary extension?

Why it matters: The VS Code extension consumes the `gert serve` JSON-RPC 2.0 API over stdio. If v2 changes the method names, parameter shapes, or notification payloads, the extension must be updated simultaneously. A versioned negotiation protocol (client declares supported API version at handshake) allows the extension to evolve independently of the binary.

Options: (a) Hard break: v2 defines a new JSON-RPC method set; the VS Code extension is updated as part of the v2 launch. (b) Versioned negotiation: the `initialize` handshake includes a protocol version field; the host advertises capabilities and the client selects a compatible version. (c) No change: the v1 JSON-RPC surface is preserved unchanged in v2.

Information needed: A complete method inventory of the v1 JSON-RPC API surface (all methods and notification shapes currently used by the VS Code extension), and a determination of which methods require changes to expose v2-specific features (e.g., saga events, OTel trace IDs).

Q7. What is the extension supply chain and signing model?

Why it matters: Extensions run in separate processes but are granted capabilities that include tool registration, schema field contribution, and event subscription. A malicious or tampered extension binary could exfiltrate captured secrets, inject commands, or suppress governance events. An extension signing model (verified publisher key) raises the bar for supply chain attacks, but adds operational friction for local development.

Options: (a) No signing required; extensions are trusted if they are present in the declared extension directory. (b) Optional signing: extensions may carry a detached signature; the host warns but does not block unsigned extensions. (c) Mandatory signing for capability classes above a threshold (e.g., extensions that request `governance.policy` capability must be signed). (d) Keyless signing via Sigstore/cosign for developer convenience.

Information needed: A threat model for the extension trust boundary (see Q4 above), and a survey of whether the target operator community has PKI infrastructure that could issue extension signing keys.

Q8. Is a gRPC transport for the VS Code extension viable for v0, or should it be deferred?

Why it matters: The current JSON-RPC 2.0 over stdio transport works but limits the VS Code extension to a single active connection and requires the extension to manage the gert process lifecycle. A gRPC transport would enable streaming (server-push events without polling), multiple concurrent clients, and connection multiplexing. However, it adds protocol complexity and requires a port allocation model.

Options: (a) Retain JSON-RPC stdio as the sole transport in v2; gRPC deferred to a future version. (b) Add gRPC as an opt-in alternative transport alongside JSON-RPC stdio. (c) Replace JSON-RPC stdio with gRPC as the primary VS Code transport; provide a backward-compatible shim for the transition period.

Information needed: Benchmarking of event delivery latency over JSON-RPC stdio versus gRPC under realistic VS Code workloads (step execution at 100ms/step, TUI

refresh at 60fps equivalent). Also requires a determination of whether VS Code's Node.js extension host supports gRPC client libraries without native module issues.

Q9. What is the parallel execution model for independent steps within a single run?

Why it matters: v1 executes all steps sequentially. Many real runbooks contain independent diagnostic steps that could execute in parallel, reducing total wall-clock time. However, parallelism complicates the trace ordering guarantee, the governance enforcer (can two parallel steps both consume the same approval token?), and the TUI rendering model.

Options: (a) No parallelism in v2; fan-out deferred to a future version. (b) Explicit parallel blocks via a new `parallel:` step type that joins on completion of all children. (c) Implicit parallelism inferred from the step dependency graph (steps with no data dependency on preceding steps execute concurrently).

Information needed: Whether any v1 runbooks in active use have independent steps that would benefit from parallelism, and what the governance implications are for concurrent step execution (e.g., two CLI steps executing simultaneously under the same allowlist).

Q10. How should the saga compensation registry interact with the trace and approval model?

Why it matters: A compensation registered for a step must itself be subject to governance (it may execute privileged commands), must appear in the trace (compensations are audit events), and may require its own approval gate if the original step had one. The interaction between the compensation executor and the governance layer is non-trivial and must be specified before implementation.

Options: (a) Compensations execute with the same governance context as the original step (same allowlist, same approver roles). (b) Compensations are exempt from approval gates (compensations are emergency rollback actions; blocking them on approval would defeat their purpose). (c) Compensation governance is separately configurable per compensation definition.

Information needed: Industry precedent from Temporal (compensation activities run in the same workflow context) and AWS Step Functions [1] (catch/retry blocks have no separate IAM policy). A decision here feeds into §?? (compensation field design) and §?? (governance scope).

Q11. What is the schema namespace convention for extension-contributed fields, and how are they validated?

Why it matters: The v2 schema design (§??) states that extension fields are namespaced, but the namespace convention and validation mechanism are undefined. If two extensions contribute fields under the same namespace, or if the host does not validate extension-contributed fields against a schema, runbooks using those fields will have unpredictable behavior at runtime.

Options: (a) Extension ID as namespace prefix (`x-myext.fieldName`); validated against a JSON Schema fragment published by the extension at registration time. (b) Dot-notation namespace under a reserved top-level key (`extensions.myext.fieldName`); host validates at parse time. (c) Extension fields are opaque to the host; the extension validates its own fields at execution time.

Information needed: Whether JSON Schema Draft 2020-12 supports dynamic `$ref` resolution at validation time, which would allow the host to assemble a composite schema from the host schema plus all registered extension schema fragments.

Q12. What is the timeout and escalation model for human approval steps?

Why it matters: v1 approval gates block indefinitely waiting for a role-holder to approve. In production, a stalled approval can block an incident response indefinitely. SLA enforcement—timeout after N minutes, escalate to the next role, optionally auto-approve or auto-reject—is a standard pattern in ITIL emergency change processes and is implemented by PagerDuty [17] and AWS Step Functions [1] human approval tasks. Without it, gert is not usable for SLA-bound incident response (*Dennis Research Brief §5.2*).

Options: (a) No timeout in v2; approval escalation deferred. (b) Optional `timeout` field on approval steps; on expiry, run is suspended and operator is notified. (c) Full escalation chain: `timeout` triggers escalation to a secondary role list; if that also times out, the step enters a configurable terminal state (auto-approve, auto-reject, or suspend-for-manual-review).

Information needed: Whether the v2 runtime concurrency model (Q2 above) supports timer-based step suspension without blocking the process, and what notification mechanism is available to alert escalation targets (stdout, webhook, or an extension-contributed notifier).

Chapter 11

Migration and Compatibility

Gert v2 is a breaking redesign from v1. This section specifies the migration path from `runbook/v0` and `runbook/v1` to the v2 schema, defines what ‘gert migrate’ (v2 version) must accomplish, and establishes compatibility guarantees for existing runbooks.

Migration is a launch blocker for adoption: every operator with a v1 runbook must understand what breaks, what is automatically migrated, and what requires manual effort.

11.1 TODO

This section is under active authoring.

Chapter 12

Governance and Policy

Governance is Gert’s core differentiator: allowlists, denylists, environment variable blocking, output redaction, and approval gates. This section defines the v2 governance model, specifying what policy fields survive from v1, how policy is evaluated at runtime, whether policy is host-enforced or extension-contributed, and how approval gates integrate with the event model.

Governance features are a first-class design requirement and must not be treated as optional or defer-to-extension.

12.1 Governance Primitives

The following primitives form gert’s policy vocabulary. Each is defined precisely below, with its evaluation point in the execution pipeline, its schema representation, and its trace event.

12.1.1 Command Allowlist (`allowed_commands`)

Definition. A whitelist of executable names (`argv[0]`, without path components) that may be invoked by `cli` steps and tool subprocess invocations. If the allowlist is non-empty, any command whose base name is not in the list is blocked before the subprocess is forked.

Semantics. Path-stripping is applied before matching: `/usr/bin/curl` matches the allowlist entry `curl`. Glob patterns are *not* supported in v2.0 (exact match only). The empty list (`[]`) means all commands are permitted (permissive default).

Schema.

```
meta:
  governance:
    allowed_commands: [kubectl, curl, jq, aws]
```

Evaluation point. Runtime Core, immediately before subprocess fork (after template evaluation). See §12.2.

Trace event. `governance/commandBlocked` — written when a command is denied. Fields: `stepId`, `command`, `reason`: `"not_in_allowlist"`.

12.1.2 Command Denylist (`denied_commands`)

Definition. An explicit list of executable names that are unconditionally blocked. The denylist takes precedence over the allowlist: if a command appears in both, it is denied.

Use case. Operators use the denylist to forbid dangerous commands (`rm`, `dd`, `shutdown`) without needing to enumerate all permitted commands.

Schema.

```
meta:
  governance:
    denied_commands: [rm, dd, shutdown, reboot]
```

Evaluation point. Runtime Core, same checkpoint as allowlist. Denylist check runs first.

Trace event. `governance/commandBlocked` — fields: `stepId`, `command`, `reason`: `"in_denylist"`.

12.1.3 Environment Variable Blocking (`deny_env_vars`)

Definition. A list of glob patterns matched against environment variable names. Any environment variable whose name matches a pattern is stripped from the subprocess environment before the command is executed.

Rationale. Prevents accidental or malicious exfiltration of secrets (e.g., `AWS_SECRET_ACCESS_KEY`, `GITHUB_TOKEN`) by commands in the runbook.

Schema.

```
meta:
  governance:
    deny_env_vars:
      - "*_SECRET*"
      - "*_TOKEN"
      - "AWS_*"
      - "GITHUB_*"
```

Evaluation point. Runtime Core, immediately before subprocess fork. Applied to the full process environment (not just explicitly passed vars).

Trace event. `governance/envVarBlocked` — fields: `stepId`, `varNames`: `[string]` (names of blocked variables, not values).

12.1.4 Output Redaction (redact)

Definition. A list of redaction rules. Each rule has a `pattern` (a RE2 regular expression) and a `replace` string. After a `cli` or `tool` step produces output, all redaction rules are applied to the captured stdout/stderr before the output is stored in run variables or written to the trace.

Semantics. Rules are applied in declaration order. The replace string may use RE2 backreferences (`$1`). The special replace value `[REDACTED]` is the conventional replacement for secrets.

Schema.

```
meta:
  governance:
    redact:
      - pattern: "token=[A-Za-z0-9_\\-]{20,}"
        replace: "token=[REDACTED]"
      - pattern: "password: .+"
        replace: "password: [REDACTED]"
      - pattern: "(?i)bearer\\s+[A-Za-z0-9._\\-]+"
        replace: "Bearer [REDACTED]"
```

Evaluation point. Runtime Core, immediately after subprocess/tool output is captured, before capture variables are set and before any trace write.

Trace event. `governance/redactionApplied` — fields: `stepId`, `ruleCount` (number of rules that matched, not the patterns themselves, to avoid logging the pattern that matched sensitive data).

12.1.5 Approval Gates

Approval gates are first-class step-level governance primitives. They are defined on any step type and pause execution until a minimum number of approvals from specified roles have been recorded. See §12.3 for the complete approval gate design.

12.2 Policy Evaluation Model

Policy primitives are evaluated at specific, well-defined points in the execution pipeline. The evaluation order within each checkpoint is fixed and must not be reordered by extensions.

Checkpoint	Primitives Evaluated	Phase
Parser validation	Schema structural rules	Parse
Planner validation	Import cycles, tool existence	Plan
Run pre-flight	Extension policy rules	Run init
Step pre-flight	Denylist check	Runtime Core
Step pre-flight	Allowlist check	Runtime Core
Step pre-flight	Approval gate	Runtime Core
Subprocess fork	Env var blocking	Runtime Core
Output captured	Redaction	Runtime Core
Post-step	Contract risk warning	Runtime Core

Order of precedence. Within the pre-flight checkpoint for a `cli` or `tool` step, the evaluation order is:

1. Denylist check (hard block, step fails immediately if matched).
2. Allowlist check (hard block, step fails immediately if not matched and list is non-empty).
3. Approval gate (soft pause; step is held until approvals arrive).
4. Env var blocking (applied at fork time, not a failure condition — silently strips).

A policy violation at any hard-block checkpoint writes the corresponding trace event and returns an error from `RunHandle.Next()`. The run is placed in the `policy_violation` terminal state. The operator must resolve the policy (e.g., add the command to the allowlist) and re-run; resumption from a policy violation is not supported in v2.0.

12.3 Approval Gate Design

12.3.1 Schema Declaration

Approval gates are declared in the step's `approvals` block:

```

steps:
- id: delete_database
  kind: cli
  command: "kubectl delete namespace {{ .targetNamespace }}"
  approvals:
    min: 2
    roles: ["DRI", "change-manager"]
    timeout: 24h
    on_timeout: fail # or: escalate
    escalate_to: ["senior-sre", "director-of-eng"]
    message: "Deleting {{ .targetNamespace }} is irreversible. Confirm change window."

```

12.3.2 UX Contract

When the Runtime Core reaches a step with an `approvals` block, it:

1. Writes `governance/approvalRequired` trace event (stepId, message, roles, min, timeout).
2. Emits an `ApprovalRequired` runtime event on the event channel.
3. Returns from `Next()` with a `StepResult` of kind `PendingApproval`. The step does *not* execute yet.
4. Subsequent calls to `Next()` on the same step return `ErrAwaitingApproval` until the approval quorum is met.

The adapter (VS Code extension, TUI, or `gert serve` client) is responsible for presenting the approval request to authorized users and collecting decisions.

12.3.3 Approval Decision Protocol

An approver submits a decision by calling `RunHandle.Approve(ctx, ApprovalDecision)`:

```

type ApprovalDecision struct {
  StepID  string // step to approve (must match current paused step)
  Actor   string // identity of the approver (e.g., "alice@corp.com")
  Role    string // role the approver is acting in (must be in step.approvals.roles)
  Approved bool    // true = approved, false = rejected

```

```

    Comment string    // optional justification
    At      time.Time // timestamp of decision
}

```

The runtime validates:

- The **Actor** identity is established (either from **-as** flag or, in a multi-user context, from the adapter's authenticated session).
- The **Role** is in the step's declared **approvals.roles**.
- The same actor may not approve the same gate twice.
- If **Approved = false**, the gate is immediately rejected (any single rejection vetos the gate), and the run enters the **rejected** terminal state.

When the approval count reaches **min**, the runtime writes **governance/approvalGranted** to the trace and the next call to **Next()** proceeds with normal step execution.

12.3.4 Approval Recording in the Trace

Every approval decision is written to the trace as an immutable record:

```

// governance/approvalRequired
{
  "seq": 14,
  "event": "governance/approvalRequired",
  "stepId": "delete_database",
  "message": "Deleting prod-ns is irreversible. Confirm change window.",
  "roles": ["DRI", "change-manager"],
  "min": 2,
  "timeout": "24h",
  "at": "2026-04-18T10:00:00Z"
}

// governance/approvalDecision (written once per decision)
{
  "seq": 15,
  "event": "governance/approvalDecision",

```

```
"stepId": "delete_database",
"actor": "alice@corp.com",
"role": "DRI",
"approved": true,
"comment": "Change window confirmed with customer.",
"at": "2026-04-18T10:05:00Z"
}

// governance/approvalGranted (written when quorum is met)
{
  "seq": 16,
  "event": "governance/approvalGranted",
  "stepId": "delete_database",
  "approvals": ["alice@corp.com", "bob@corp.com"],
  "at": "2026-04-18T10:07:00Z"
}
```

The trace provides a complete, tamper-evident audit record of who approved what and when.

12.3.5 Timeout and Escalation

If `timeout` is set and the approval quorum is not met within the deadline:

- `on_timeout: fail`: The run enters the `timed_out` terminal state. `governance/approvalTimedOut` is written to the trace.
- `on_timeout: escalate`: The runtime emits an `ApprovalEscalated` event. If `escalate_to` roles are declared, a new `ApprovalRequired` event is emitted targeting those roles. The timeout clock restarts. Escalation is attempted once; if the escalated request also times out, the run fails.

Timeout enforcement is implemented via a `context.WithDeadline` passed to the approval wait loop. The adapter (or the `gert serve` session timer) is responsible for surfacing the timeout to the approver.

12.4 Policy-as-Code Direction

12.4.1 v2.0: Structured YAML Policy Blocks

For v2.0, gert uses **structured YAML policy blocks** embedded in the runbook's `meta.governance` section. This is the approach inherited from v1, extended with the richer primitives defined in §12.3.

Rationale for v2.0:

- The policy vocabulary is small and closed (six primitives as of v2.0). A full policy DSL would be over-engineered for this set.
- YAML policy blocks are self-contained: the runbook declares its own governance constraints without requiring external policy files or a running policy server.
- Structured blocks are straightforward to validate with JSON Schema at authoring time, giving operators early feedback in the VS Code extension.
- The existing v1 governance model (allowlists, redaction, env blocking) translates directly to structured blocks with no loss of expressiveness.

12.4.2 v2.1+: OPA Integration Hooks

Dennis's research confirms that Open Policy Agent (OPA) is the CNCF standard for policy-as-code in operational tooling. For **v2.1**, gert will introduce OPA integration at the following evaluation hooks:

```
// OPA evaluation hook interface (v2.1 target)
type PolicyEngine interface {
    // EvaluatePreStep is called before each step pre-flight.
    // The policy engine receives the full step context and returns
    // allow/deny with a reason.
    EvaluatePreStep(ctx context.Context, input PolicyInput) (PolicyDecision, error)

    // EvaluatePreRun is called once after plan construction.
    // Used for runbook-level access control (RBAC).
    EvaluatePreRun(ctx context.Context, input PolicyInput) (PolicyDecision, error)
}
```

```
type PolicyInput struct {
    Runbook  RunbookMeta
    Step     ResolvedStep
    Actor    string
    Vars     map[string]string
    Contract *contract.Contract
}

type PolicyDecision struct {
    Allow bool
    Reason string
    // For approval gates: required roles computed by policy engine
    RequiredApprovers []string
}
```

The PolicyEngine interface will have two implementations in v2.1:

1. **Built-in:** evaluates the structured YAML policy blocks (backward-compatible with v2.0 runbooks).
2. **OPA:** calls an OPA bundle loaded from a project policy directory (`.gert/policy/*.rego`).

The two implementations compose: the built-in engine runs first; if it allows, the OPA engine runs second. A denial from either blocks the step.

Recommendation: Do not block the v2.0 launch on OPA integration. Ship the structured YAML model first, then introduce OPA as an opt-in engine in v2.1. This allows the governance interface to be designed correctly without the complexity of Rego authoring in the initial release.

12.5 RBAC Model

12.5.1 Identity Establishment

Actor identity is established via the `-as <identity>` flag passed at invocation time. The identity string is an opaque label (conventionally an email address or a role name) recorded in the trace and used for approval gate matching.

v2.0 limitation: Identity is not authenticated. The `-as` flag is self-asserted; gert does not verify that the caller is who they claim to be. Enforcement of identity authenticity is

the responsibility of the surrounding infrastructure (CI/CD pipeline, VS Code extension SSO, MCP server authentication).

v2.1 target: An `IdentityProvider` interface will allow adapters to supply verified identity tokens (e.g., OIDC tokens from VS Code GitHub Copilot authentication), replacing self-asserted identity for governed environments.

12.5.2 Runbook-Level Access Control

Runbooks may declare a `requires_role` field to restrict who may execute them:

```
meta:
  governance:
    requires_role: "incident-responder"
    # or: requires_any_role: ["DRI", "incident-responder"]
    # or: requires_all_roles: ["DRI", "change-manager"]
```

The Runtime Core evaluates this constraint during run initialization (pre-flight, before any step executes). If the actor's identity does not match, `governance/accessDenied` is written to the trace and the run fails immediately.

12.5.3 Step-Level Role Restriction

Individual steps may declare `requires_role` to restrict who may trigger that step. This is distinct from approval gates (which require external approval) — step role restriction *prevents* a step from executing unless the actor themselves holds the required role.

```
steps:
  - id: promote_to_production
    kind: cli
    requires_role: "release-manager"
    command: "kubectl set image deployment/app app={{ .imageTag }}"
```

12.5.4 Separation of Duties

The approval gate model supports separation of duties: the `same-actor-cannot-approve` invariant (defined in §12.3) ensures that a step author cannot self-approve their own gate. In environments requiring formal change management, the actor who initiates the run and the actor(s) who approve high-risk steps must be different.

12.6 Redaction

12.6.1 Pattern-Based Redaction

The primary redaction mechanism is pattern-based: operators declare RE2 regular expressions in `meta.governance.redact`. Redaction is applied eagerly: captured output is redacted *before* any capture variable assignment or trace write.

12.6.2 Explicit Field Marking (v2.1)

For structured tool outputs (JSON objects returned by MCP tools), v2.1 will introduce explicit field marking: a tool definition may declare that a returned field contains sensitive data. The runtime will redact the field value without requiring a regex pattern.

```
# In .tool.yaml (v2.1 target)
outputs:
  - name: api_key
    sensitive: true    # value redacted from trace and captures
```

12.6.3 Redaction Guarantees

- Redaction is applied to all output paths: captured variables, trace events, and event channel payloads.
- Redaction rules are compiled (regex pre-compiled) during run initialization, not re-compiled per step.
- Redaction is applied unconditionally in all run modes (real, dry-run, replay).
- The original (pre-redaction) value is *never* written to disk in any path.

12.7 Audit Trail Requirements

The append-only JSONL trace file is gert's primary audit artifact. Every governance event is a first-class trace entry. The following governance events are guaranteed to be written:

Trace Event	Written When
<code>governance/commandBlocked</code>	A command is blocked by denylist or allowlist
<code>governance/envVarBlocked</code>	Env vars are stripped before subprocess fork
<code>governance/redactionApplied</code>	Redaction rules matched captured output
<code>governance/approvalRequired</code>	A step's approval gate is entered
<code>governance/approvalDecision</code>	An approver submits approve or reject
<code>governance/approvalGranted</code>	Approval quorum is met
<code>governance/approvalRejected</code>	An approver rejects the gate
<code>governance/approvalTimedOut</code>	Approval timeout expires
<code>governance/accessDenied</code>	RBAC check fails (run or step level)
<code>governance/contractWarning</code>	Step contract risk level is critical
<code>governance/policyViolation</code>	Generic policy violation (OPA engine, v2.1)

12.7.1 Trace Integrity

The trace file is protected by:

- **Append-only write discipline:** the trace writer never seeks backward or truncates. Each write is followed by `fsync`.
- **SHA256 evidence hash:** at run completion, the SHA256 of the entire trace file is written as the final record and also stored in a sidecar `trace.sha256` file.
- **Monotonic sequence numbers:** every trace event has a `seq` field (incrementing integer). Gaps in sequence numbers signal corruption or tampering.

12.7.2 Compliance Alignment

These audit trail guarantees are designed to satisfy the requirements of:

- **SOC 2 Type II** — Change management controls: every command execution and approval is logged with actor identity and timestamp.
- **HIPAA** — Access controls and audit logging: env var blocking and redaction prevent PHI from appearing in traces; approval gates enforce separation of duties.
- **ITIL Change Management** — Normal/standard/emergency change paths can be modeled via `requires_role` and multi-approver gates.

12.8 Extension-Contributed Policy

Extensions may contribute governance rules through the `ContributedPolicyRules()` method of the Extension Host (see §02). These rules compose with host policy under the following invariants:

1. **Additive only.** Extension rules may add restrictions; they may never remove or override host-defined policy. An extension cannot remove a command from the denylist, relax a required role, or suppress a redaction rule.
2. **Host policy takes precedence on conflict.** If an extension rule and a host rule address the same primitive and disagree, the more restrictive interpretation applies.
3. **Extension rules are visible in the trace.** A `governance/extensionPolicyApplied` trace event is written when an extension-contributed rule is evaluated, naming the extension and rule ID.
4. **Extension rules are not trusted with approval authority.** Extensions may not grant approvals on behalf of human approvers. Approval decisions must originate from a human actor via the `RunHandle.Approve()` API.

12.8.1 Extension Policy Rule Schema

```
// PolicyRule is contributed by an extension and evaluated at pre-step checkpoint.
type PolicyRule struct {
    ID          string          // unique within the extension's namespace
    Description string
    // Evaluate returns Deny if the rule blocks this step, Allow otherwise.
    // Extensions communicate rules as static declarations (v2.0); dynamic
    // rule evaluation via the extension RPC transport is v2.1.
    StepKinds []StepKind // step types this rule applies to (empty = all)
    CommandGlob string      // if set, only applies to commands matching this glob
    Action     PolicyAction // Deny or Warn
    Reason     string        // human-readable reason written to trace event
}
```

Extension-contributed policy is loaded during the Extension Host's `Load()` phase, before the first `Next()` call. Policy rules are static for the lifetime of a run; extensions may not add or remove rules after the run has started.

Chapter 13

Evidence, Tracing, and Resumption

Gert v2 captures a complete, durable record of every execution for audit, compliance, incident response, and operational replay. This section specifies the append-only JSONL trace format, evidence capture model, per-step snapshot contract, run resumption semantics, replay mode, and OpenTelemetry integration. Together these mechanisms ensure that every run produces a self-contained artifact that answers three questions: *what happened*, *what evidence was collected*, and *how can execution be recovered or replicated*.

13.1 Trace File Format

13.1.1 File Location and Naming

Each run produces a single trace file located at:

```
.runbook/runs/<run-id>/trace.jsonl
```

The `<run-id>` is a URL-safe string assigned at run creation time (e.g. `20260418T143022-a3f9c1`). All run artefacts are co-located under the run directory:

```
.runbook/runs/<run-id>/
  trace.jsonl           # append-only JSONL event log
  snapshots/           # per-step state checkpoints
    step-0000.json
    step-0001.json
    ...
  attachments/         # binary evidence referenced from trace events
    <sha256>.bin
```

```

...
run.yaml          # completion manifest (written on terminal event)
annotations.jsonl # operator notes (append-only)
lock              # PID lock (removed on clean exit)

```

13.1.2 JSONL Envelope

Every line in `trace.jsonl` is a self-contained JSON object with a common envelope. Consumers MUST tolerate unknown fields to allow additive schema evolution.

```

{
  "seq":      <int64>,    // monotonically increasing within this run
  "type":     <string>,    // event type (see §12.1.3)
  "timestamp": <RFC3339>,  // UTC timestamp with sub-second precision
  "run_id":   <string>,    // run identifier
  "path":     <array>,     // tree-path to the active node (may be null)
  "data":     <object>     // type-specific payload (see below)
}

```

The `seq` field provides a total order over events in a single run. It is assigned by the `RunStore.WriteTrace` implementation and is not trusted for cross-run ordering.

13.1.3 Event Type Catalogue

The following event types are normative in v2. All user-supplied string fields (command arguments, outputs, prompt responses) pass through the redaction pipeline before being written.

`run/started`

Emitted once, as the first event in every trace file.

```

"data": {
  "runbook_id": <string>, // runbook name/path
  "user":       <string>,  // actor identity
  "mode":       <string>,  // "real" | "replay" | "dry-run"
  "inputs":     <object>,  // resolved input values (redacted)
}

```

```
"client":      <string>, // "cli" | "tui" | "web" | "mcp"
"parent_run_id": <string> // omitted if not an invoke child
}
```

step/started

Emitted immediately before executing a step.

```
"data": {
  "step_id": <string>, // stable step identifier
  "step_type": <string>, // "cli" | "manual" | "tool" | "invoke"
  "step_name": <string> // human-readable name from runbook YAML
}
```

step/completed

Emitted after a step exits cleanly.

```
"data": {
  "step_id": <string>,
  "exit_status": <int>, // exit code for cli steps; 0 for others
  "duration_ms": <int64>, // wall-clock duration in milliseconds
  "captures": <object>, // captured output variables (redacted)
  "evidence": [ // zero or more evidence items
    {
      "name": <string>,
      "kind": "text" | "checklist" | "attachment",
      "value": <string>, // for kind=text
      "items": <object>, // for kind=checklist
      "path": <string>, // relative path for kind=attachment
      "sha256": <string>, // hex SHA256 for kind=attachment
      "size": <int64> // bytes for kind=attachment
    }
  ]
}
```

step/failed

Emitted when a step exits with a non-zero status or returns a runtime error.

```
"data": {
  "step_id":    <string>,
  "error_type": <string>, // "exit_code" | "timeout" | "governance"
                                     // | "tool_error" | "invoke_failed"
  "error":      <string>, // human-readable message (redacted)
  "exit_status": <int>,   // for error_type=exit_code
  "duration_ms": <int64>
}
```

step/skipped

Emitted when a step is not executed due to branch evaluation or iterate exhaustion.

```
"data": {
  "step_id": <string>,
  "reason":  "branch_not_taken" | "iterate_exhausted" | "dry_run"
}
```

step/retrying

Emitted before each retry attempt (requires the retry policy feature).

```
"data": {
  "step_id":    <string>,
  "attempt":    <int>,    // attempt number (1 = first retry)
  "max_attempts": <int>,
  "backoff_ms":  <int64>
}
```

tool/invoked

Emitted when the tool runtime dispatches a tool call.

```
"data": {
  "tool_name":      <string>,
  "transport":      "stdio" | "jsonrpc" | "mcp",
  "correlation_id": <string>, // UUIDv4, links to tool/responded
  "inputs":         <object>  // sanitized inputs
}
```

tool/responded

Emitted when the tool call returns.

```
"data": {
  "correlation_id": <string>,
  "exit_code":      <int>,
  "duration_ms":    <int64>,
  "truncated":      <bool>    // true if output was capped
}
```

manual/prompted

Emitted when a manual step displays its prompt to the operator.

```
"data": {
  "step_id":      <string>,
  "prompt_text":  <string>, // the prompt shown (redacted)
  "sla_seconds":  <int>     // 0 if no SLA configured
}
```

manual/completed

Emitted when the operator submits a response to a manual step.

```
"data": {
  "step_id":      <string>,
  "response":      <string>, // operator's text response (redacted)
  "attestation":   <string>, // "operator_confirmed" | "skipped"
  "approver":      <string>, // identity of responding actor
  "duration_ms":   <int64>
}
```

governance/blocked

Emitted when a governance rule blocks execution.

```
"data": {
  "step_id":      <string>,
  "rule":         <string>, // allowlist | denylist | env_block
                                // | output_redact | approval_gate
  "action_attempted": <string>, // description of blocked action
  "policy":       <string> // policy name or path
}
```

governance/approved

Emitted when an approval gate is cleared.

```
"data": {
  "step_id":      <string>,
  "approver":     <string>, // identity of approver
  "approval_method": <string>, // "interactive" | "mcp" | "api"
  "duration_ms":  <int64>
}
```

run/completed

Terminal event for a successful run.

```
"data": {
  "final_status": "passed" | "failed" | "skipped",
  "duration_ms":  <int64>,
  "step_count":   <int>,
  "passed":       <int>,
  "failed":       <int>,
  "skipped":      <int>
}
```

run/failed

Terminal event for a run that halted due to an unrecoverable error.

```
"data": {
  "failing_step_id": <string>,
  "error":           <string>,
  "duration_ms":     <int64>
}
```

checkpoint

Internal event written by `RunStore.SaveCheckpoint` to link a snapshot file to the trace sequence. Not user-facing but required for resumption.

```
"data": {
  "snapshot_file": <string> // e.g. "step-0003.json"
}
```

13.1.4 Crash Safety

The trace file is opened with `O_APPEND` and each event is written as a single `write(2)` call followed by `fsync`. On POSIX systems, a write of fewer than `PIPE_BUF` bytes (4096 bytes on Linux) to an `O_APPEND` file is atomic. Because the JSON encoding of a single trace event is always well under 4096 bytes for typical runbooks, partial-line corruption on crash is avoided in practice. For large payloads (e.g. tool output), the runtime truncates at a configurable limit (default 4 KB per field) before encoding.

The Go implementation:

```
// WriteTrace appends one event atomically and fsyncs.
func (d *DirRunStore) WriteTrace(event DurableEvent) error {
    p := filepath.Join(d.baseDir, "trace.jsonl")
    f, err := os.OpenFile(p, os.O_CREATE|os.O_WRONLY|os.O_APPEND, 0644)
    if err != nil {
        return fmt.Errorf("open trace: %w", err)
    }
    defer f.Close()
```

```
data, err := json.Marshal(event)
if err != nil {
    return fmt.Errorf("marshal event: %w", err)
}
data = append(data, '\n') // JSONL delimiter
if _, err := f.Write(data); err != nil {
    return fmt.Errorf("write trace: %w", err)
}
return f.Sync() // durability guarantee
}
```

On recovery, any line that fails `json.Unmarshal` is silently skipped. The last clean `checkpoint` event determines the recoverable state.

13.1.5 Tamper Evidence

The v2 trace format supports optional HMAC-SHA256 signing of each event. When a signing key is configured (via `GERT_TRACE_KEY` or the workspace config), the runtime appends a `"sig"` field to the envelope containing the HMAC-SHA256 of the canonical JSON of all other fields. Verifiers can replay the trace and recompute signatures. This does not prevent deletion of events (append-only at the filesystem level provides that property) but detects modification of existing events.

13.2 Evidence Capture

13.2.1 What Constitutes Evidence

In v2, “evidence” is any artefact that attests to a step’s outcome:

- **Command output** — `stdout/stderr` of a `cli` step, captured and embedded in `step/completed`.
- **Manual attestations** — operator-provided text, checklist responses, and explicit confirmations from `manual` steps.
- **Tool responses** — structured JSON returned by a `tool` step, captured in `step/completed.captures`.

- **File attachments** — binary or text files explicitly attached by an operator or produced by a step, stored in the `attachments/` directory and referenced by SHA256 digest.

13.2.2 SHA256 Hashing

All file attachments are content-addressed by their SHA256 digest. The `evidence` package exposes:

```
// HashFile returns hex SHA256 and file size.
func HashFile(path string) (sha256hex string, size int64, err error)

// NewAttachmentEvidence creates an EvidenceValue with hash and size.
func NewAttachmentEvidence(path string) (*EvidenceValue, error)
```

The hash is stored in the `step/completed` event alongside the relative path under `attachments/`. Consumers can verify integrity at any time by re-hashing the referenced file.

Text evidence fields (command output, operator responses) are not individually hashed — the overall trace event integrity is provided by the optional HMAC-SHA256 event signature described in § 12.1.5.

13.2.3 File Attachments

When a step produces a binary artefact, the runtime:

1. Copies the file to `.runbook/runs/<run-id>/attachments/<sha256>.<ext>`.
2. Emits an `EvidenceValue` of kind `attachment` in the `step/completed` event with fields `path`, `sha256`, and `size`.

Attachments are deduplicated by content: if two steps produce identical file content, they share one attachment file. The trace events may reference the same `sha256` value independently.

13.2.4 Evidence Retention

Evidence is retained for the lifetime of the run directory. The workspace-level retention policy (configurable in `gert.yaml`) specifies:

- `retention.runs` — number of completed runs to retain (default: 50).
- `retention.max_age_days` — maximum age in days (default: 90).
- `retention.attachments` — whether to include attachments in retention (default: true).

The `gert gc` command enforces retention policy by removing the oldest run directories, deleting their `attachments/` content first.

13.3 Run State Snapshots

13.3.1 Checkpoint Model

A checkpoint is a complete serialisation of `RunState` written to `snapshots/step-NNNN.json` after every successfully completed step. The snapshot captures the minimum state needed to resume execution from the next step:

```
type RunState struct {
    RunID                string
    RunbookPath          string
    Mode                 string           // "real" | "replay" | "dry-run"
    StartedAt            time.Time
    Actor                string
    CurrentStepIndex     int             // flat-mode index (legacy compat)
    Path                 TreePath        // tree-mode cursor (authoritative)
    Vars                  map[string]string // resolved input bindings
    Captures              map[string]string // accumulated step outputs
    History               []*StepResult    // completed step records
    InvokeStack           []InvokeFrame    // nested invoke context stack
    IterateState          map[string]*IterateProgress
    ParallelSlots         map[string][]SlotState
    LastCheckpointSeq     int64           // trace seq at checkpoint time
}
```

13.3.2 Atomic Snapshot Write

Snapshots are written atomically: the runtime marshals the state to a `.tmp` file, fsyncs it, then renames it to the final path. An incomplete snapshot (e.g. from a crash mid-write) retains the `.tmp` suffix and is skipped during recovery.

```
func (d *DirRunStore) SaveCheckpoint(state *RunState) (string, error) {
    filename := fmt.Sprintf("step-%04d.json", len(state.History))
    finalPath := filepath.Join(d.baseDir, "snapshots", filename)
    tmpPath   := finalPath + ".tmp"

    data, _ := json.MarshalIndent(state, "", " ")
    os.WriteFile(tmpPath, data, 0644)           // write to temp
    tf, _ := os.Open(tmpPath); tf.Sync(); tf.Close() // fsync temp
    os.Rename(tmpPath, finalPath)                // atomic rename
    // ... then write checkpoint trace event
}
```

13.3.3 Minimum Resumption State

The state required to resume a run from step *N* is exactly the snapshot at `step-NNNN.json`. It contains:

- All resolved input values (**Vars**).
- Outputs captured by completed steps (**Captures**).
- The tree cursor (**Path**) identifying which node executes next.
- Branch decisions already taken (encoded in **History** step statuses).
- Iterate progress counters and parallel slot states.
- The invoke stack for nested runbooks.

The runbook YAML itself is not stored in the snapshot; it is re-read from `RunbookPath` at resume time. Operators must not modify the runbook between the original run and resumption.

13.4 Run Resumption

13.4.1 The `gert exec -resume` Contract

`gert exec --resume <run-id>`

The runtime executes the following recovery sequence:

1. Locate `.runbook/runs/<run-id>/`.
2. Attempt to acquire the `lock` file. Fail if a live process holds it.
3. Call `RunStore.LoadLatestCheckpoint()` — scan `snapshots/` descending, skip `.tmp` files, return the newest valid JSON.
4. Re-open `trace.jsonl` for append.
5. Advance `CurrentStepIndex` by one (or advance the tree `Path` cursor past the last completed node).
6. Re-build the governance engine from the runbook’s policy block.
7. Resume the execution loop from the restored `Path`.

The resumption does *not* re-execute any step recorded in `History`. It continues from the step *after* the last checkpoint.

13.4.2 Resumability by Step Type

- **cli steps** — safe to resume. The step is re-executed from scratch. Runbook authors should ensure cli steps are idempotent (e.g. `kubectl apply` over `kubectl create`).
- **manual steps** — the operator is re-prompted. The prior response (if any) is lost. The prompt is re-displayed and the operator must re-attest.
- **tool steps** — resumable if idempotent. If the tool call was in-flight when the process crashed (i.e. a `tool/invoked` event exists with no matching `tool/responded`), the runtime re-issues the call. Authors must declare `idempotent: true` on tools where safe; others are re-executed with a warning log.

- **invoke steps** — if the child run completed (its terminal event is in the parent trace), the invoke is treated as completed. If the child run was in progress, the child is itself resumed via the nested run ID stored in the **InvokeStack**.
- **branch/iterate** — structural nodes; not executed directly. Cursor advancement handles re-entry correctly.

13.4.3 Idempotency Guidance

Runbook authors should write resumable runbooks by following these guidelines:

- Use `kubectl apply` and `terraform apply` instead of imperative create commands.
- Use `-if-not-exists` or `-create-or-replace` flags where available.
- For tool steps, declare `idempotent: true` in the tool definition when the operation is safe to call multiple times.
- For manual steps, write prompts as questions that remain valid on a second ask (“Has the database migration completed?” rather than “Confirm you have started the migration”).
- Avoid steps whose side effects depend on the current time or a random seed without an explicit override mechanism.

13.4.4 Partial Step Resumption

If a `tool/invoked` event has no matching `tool/responded` event, the tool call was interrupted mid-flight. On resumption:

1. The runtime logs a warning identifying the orphaned correlation ID.
2. If `idempotent: true`: the tool call is re-issued from the beginning.
3. If `idempotent: false` (default): the step is treated as **failed** and execution follows the normal failure path (error branch, if defined, or halt).

13.5 Replay Mode

13.5.1 Overview

```
gert exec --mode replay --scenario <scenario.yaml>
```

Replay mode executes a runbook against a set of pre-recorded command responses and evidence values. No real commands are executed and no real tools are invoked. The mode is used for:

- **Testing** — validating that a runbook’s branching logic, captures, and governance rules behave correctly for a given set of inputs and command outputs (see § 13 Testing and Acceptance).
- **Demonstration** — showing a runbook’s execution flow without real infrastructure.
- **Regression** — verifying that a refactored runbook produces the same trace as the original.

13.5.2 Scenario File Format

A scenario file is a YAML document that maps commands to recorded responses and step IDs to recorded evidence:

```
commands:
  - argv: ["kubectl", "get", "pods", "-n", "prod"]
    stdout: "NAME          READY  STATUS   RESTARTS\npod-abc    1/1    Running  0\n"
    stderr: ""
    exit_code: 0

evidence:
  check-pod-health:      # step_id
    observation:         # evidence name
      kind: text
      value: "pod is healthy, proceeding"
```

Commands are matched in order of declaration (first match wins). Unmatched commands fail the scenario unless `allow_unmatched: true` is set.

13.5.3 Replay Semantics

In replay mode:

- **cli** steps — the command executor returns the pre-recorded stdout/stderr/exit_code.
- **manual** steps — the evidence collector returns the pre-recorded evidence values instead of prompting the operator.
- **tool** steps — tool invocations are intercepted; if a matching recorded response exists, it is returned. Otherwise the step fails.
- **invoke** steps — the child runbook is executed in replay mode using the same scenario (or a nested scenario file if `scenario_file:` is specified on the invoke step).
- Governance rules — evaluated normally; replay does not bypass governance.
- Trace events — written normally; the replay trace can be compared against a golden trace for regression testing.

13.5.4 Determinism Requirement

For replay to produce the same outcome across runs, the runbook must be deterministic given fixed inputs and command outputs. Non-determinism sources to avoid:

- Timestamps in expressions (use captured values instead of `time.Now()`).
- Random identifiers in step names or captures.
- File system state that is not reproduced by the scenario.
- External API calls in tool steps that are not replayed (tool steps should be declared `idempotent: true` and covered by scenario fixtures).

13.6 OpenTelemetry Integration

13.6.1 Rationale

The JSONL trace format is optimised for durability, human readability, and offline analysis. OpenTelemetry (OTel) spans are optimised for real-time observability, distributed tracing, and integration with platforms such as Jaeger, Zipkin, Datadog, and Grafana Tempo. Both serve complementary purposes and v2 supports both simultaneously.

13.6.2 Span Hierarchy

When OTel is enabled, the runtime creates a span hierarchy that mirrors the runbook execution tree:

```
Span: gert.run           // root span, covers entire run
  Span: gert.step        // one per step executed
    Span: gert.tool.invoke // one per tool call within a step
  Span: gert.step        // next step
  ...
```

The Go implementation uses `go.opentelemetry.io/otel/trace`:

```
func (e *Engine) runWithOTel(ctx context.Context) error {
    ctx, runSpan := otel.Tracer("gert").Start(ctx, "gert.run",
        trace.WithAttributes(
            attribute.String("gert.run_id",    e.State.RunID),
            attribute.String("gert.runbook",   e.Runbook.Meta.Name),
            attribute.String("gert.actor",     e.State.Actor),
            attribute.String("gert.mode",      e.State.Mode),
        ),
    )
    defer runSpan.End()
    return e.execute(ctx)
}

func (e *Engine) executeStep(ctx context.Context, step Step) error {
    ctx, stepSpan := otel.Tracer("gert").Start(ctx, "gert.step",
        trace.WithAttributes(
            attribute.String("gert.step_id",    step.ID),
            attribute.String("gert.step_type",  step.Type),
            attribute.String("gert.step_name",  step.Name),
        ),
    )
    defer stepSpan.End()
    // ...
}
```

13.6.3 Span Attributes

The following attributes are set on all spans:

Attribute	Spans	Description
<code>gert.run_id</code>	all	Run identifier
<code>gert.runbook</code>	all	Runbook name
<code>gert.actor</code>	run	Actor identity
<code>gert.mode</code>	run	Execution mode
<code>gert.step_id</code>	step, tool	Step identifier
<code>gert.step_type</code>	step	Step type
<code>gert.tool_name</code>	tool	Tool name
<code>gert.transport</code>	tool	Transport (stdio/jsonrpc/mcp)

Trace context is propagated into tool invocations via the W3C `traceparent` header where the transport supports it (HTTP-based MCP tools). For stdio tools, context is passed as an environment variable `OTEL_TRACEPARENT`.

13.6.4 Relationship to JSONL Trace

OTel spans and the JSONL trace are complementary, not redundant:

- The JSONL trace is the authoritative durable record. It contains evidence hashes, governance events, snapshot references, and redacted output that OTel spans do not.
- OTel spans are ephemeral, push-based, and optimised for latency-sensitive observability dashboards.
- The trace event `"seq"` is embedded as the `gert.trace_seq` span attribute, enabling correlation between spans and trace events.

13.6.5 Configuration

OTel is opt-in and configured via standard OTEL environment variables:

```
OTEL_EXPORTER_OTLP_ENDPOINT=http://localhost:4317
OTEL_SERVICE_NAME=gert
OTEL_TRACES_SAMPLER=always_on
```

When `OTEL_EXPORTER_OTLP_ENDPOINT` is not set, the OTel SDK is initialised with a no-op tracer and zero overhead is incurred. There is no `-otel` flag; configuration is entirely via the environment. This ensures that CI and offline use work without modification.

13.7 Implementation Notes for Go

The key Go interfaces for this subsystem are:

```
// TraceWriter is the interface the engine uses to record events.
// Implementations must be safe for concurrent use from a single goroutine
// (the engine's execution loop); multi-goroutine safety is optional.
type TraceWriter interface {
    Write(event DurableEvent) error
    Close() error
}

// RunStore manages all artefacts for a single run.
type RunStore interface {
    RunID() string
    WriteTrace(event DurableEvent) error
    ReadTrace() ([]DurableEvent, error)
    ReadTraceSince(afterSeq int64) ([]DurableEvent, error)
    SaveCheckpoint(state *RunState) (string, error)
    LoadLatestCheckpoint() (*RunState, int64, error)
    AcquireLock() error
    ReleaseLock() error
    WriteManifest(manifest *RunManifest) error
}

// EvidenceCollector is the interface for gathering evidence at manual steps.
type EvidenceCollector interface {
    Collect(ctx context.Context, stepID string,
        prompts []Prompt) (map[string]*EvidenceValue, error)
}
```

The **DirRunStore** is the production implementation. A **MemRunStore** (storing events in a slice) is provided for unit tests where filesystem I/O is undesirable. The **ReplayEvidenceCollector** returns pre-recorded evidence for scenario tests.

Chapter 14

Adapter Contracts

Gert v2 decouples the core runtime from presentation adapters (TUI, Web, VS Code) through explicitly versioned contracts. This section defines the concrete interface that adapters implement: event subscription API, initialization handshake, render lifecycle, and the non-negotiable constraint that adapters own no execution logic. This contract must be stable and extensible to support current and future adapters.

14.1 TODO

This section is under active authoring.

Chapter 15

Input Provider Framework

The input provider framework governs how gert resolves dynamic input values at run time. When a runbook declares an input with a `from:` binding, the provider framework locates the appropriate provider, dispatches a resolution request, and returns the concrete value before the first step that needs it begins.

15.1 Design Goals

- **Extensibility:** third-party systems (PagerDuty, ServiceNow, Vault, etc.) integrate as provider plugins without changes to the host binary.
- **Transparency:** every resolved value is recorded in the run trace so the operator can audit where each input came from.
- **Fallback safety:** resolution failures degrade gracefully to an interactive prompt rather than aborting the run silently.
- **Consistency with tools:** providers share the same transport layer as tools (`stdio`, `stdio-jsonrpc`, `mcp`), minimising implementation surface.

15.2 Provider Definition Schema

Each provider is declared in a `.provider.yaml` file. By convention, project-level providers live in `providers/<name>.provider.yaml`.

```
# providers/pagerduty.provider.yaml
apiVersion: provider/v2
```

```
meta:
  name: pd                                # Prefix used in from: bindings (e.g. pd.*)
  version: "1.0.0"
  description: "Resolves inputs from PagerDuty incidents and services"
  binary: pd-provider                     # Executable name (looked up in PATH)

transport:
  mode: stdio-jsonrpc                     # stdio | stdio-jsonrpc | mcp
  startup:
    argv: ["--config", "pd.yaml"]
    ready-pattern: "provider ready"
    timeout: "10s"
    shutdown-method: "shutdown"

resolution:
  method: provider/resolve                 # JSON-RPC method the host calls
  prefixes:
    - "pd."                               # All from: values starting with pd.
                                           # are dispatched to this provider

# Schema for what the provider needs in each resolution request
input-schema:
  type: object
  properties:
    bindings:
      type: object
      description: "Map of input name to from: binding string"
    context:
      type: object
      description: "Execution context (runId, userId, mode)"
  required: ["bindings"]

# Schema for what the provider returns
output-schema:
  type: object
```

```
properties:
  resolved:
    type: object
    description: "Map of input name to resolved string value"
  warnings:
    type: array
    items:
      type: string
  required: ["resolved"]

cache:
  enabled: true
  scope: run                # run | step | none
  ttl: "60s"

governance:
  requires-capabilities:
    - capability/network    # This provider needs outbound network
  env-read-patterns:
    - "PD_API_KEY"
    - "PD_*
```

15.3 Resolution Protocol

15.3.1 Resolution Flow

When the run engine encounters an input with a `from:` binding (e.g., `from: pd.incident.title`), the following steps occur:

1. **Prefix match:** the framework scans registered providers for one whose declared prefix matches the start of the binding string. The longest matching prefix wins.
2. **Provider start:** if the matched provider's process is not yet running, the host spawns it and waits for the ready signal (same mechanics as `stdio-jsonrpc` tools; see Section 6.3).

3. **Batch collection:** all inputs whose binding prefix maps to the same provider are grouped into a single resolution request to minimise round-trips.
4. **JSON-RPC call:** the host sends a `provider/resolve` request.
5. **Result merge:** resolved values are merged into the run's variable map.
6. **Fallback:** any binding that was not resolved (absent from the response's `resolved` map, or where the provider returned a warning) falls back according to the input's `fallback` field (default: interactive prompt).

15.3.2 Resolution Request Envelope

```
{
  "jsonrpc": "2.0",
  "id": "prov-1",
  "method": "provider/resolve",
  "params": {
    "bindings": {
      "incident_title": "pd.incident.title",
      "incident_severity": "pd.incident.severity",
      "service_name": "pd.service.name"
    },
    "context": {
      "runId": "run-abc123",
      "stepId": "__pre-flight__",
      "userId": "ops@acme.example",
      "mode": "real",
      "traceId": "trace-xyz"
    }
  }
}
```

15.3.3 Resolution Response Envelope

```
{
  "jsonrpc": "2.0",
  "id": "prov-1",
```

```

"result": {
  "resolved": {
    "incident_title": "Database connection pool exhausted",
    "incident_severity": "P1",
    "service_name": "payments-api"
  },
  "warnings": [],
  "telemetry": {
    "startedAt": "2026-04-18T10:00:00Z",
    "finishedAt": "2026-04-18T10:00:00.080Z",
    "durationMs": 80
  }
}
}

```

15.3.4 Resolution Error Handling

If the provider returns a JSON-RPC error, or if a binding key is absent from the **resolved** map, the host applies the input's fallback strategy:

```

# In runbook meta.inputs
inputs:
  incident_title:
    from: pd.incident.title
    description: "Active incident title"
    fallback: prompt          # prompt | fail | default
    default: "Unknown incident" # Used when fallback: default

```

- **fallback: prompt** (default): the host issues an interactive prompt for the value. In non-interactive modes (dry-run, CI) this causes the run to fail.
- **fallback: default**: the declared **default:** value is used without prompting. A warning is written to the trace.
- **fallback: fail**: the run aborts immediately with a structured error.

15.3.5 Resolution Caching

Providers may declare a cache policy in their definition. The host respects this policy:

- **scope:** `run` — the resolved value is reused for all subsequent steps in the same run that reference the same binding. This is the default for external providers.
- **scope:** `step` — the value is resolved fresh on each step that references it.
- **scope:** `none` — caching is disabled; every binding triggers a new RPC call.

Cache entries are keyed by `(runId, bindingString)`. Cached values are stored in the run's in-memory variable map only; they are not persisted across run resumptions.

15.4 Built-in Providers

The following providers are compiled into the gert host binary and are always available without a `.provider.yaml` file.

15.4.1 `env`

Reads values from environment variables.

```
inputs:
  api_key:
    from: env.ACME_API_KEY
    description: "API key for the Acme service"
```

The `env.` prefix is followed by the exact environment variable name. If the variable is not set, resolution fails and the fallback strategy is applied. The `env` provider never contacts an external process; it reads directly from the host's environment.

15.4.2 `file`

Reads values from files on the local file system.

```
inputs:
  ssh_key:
```

```
    from: "file ~/.ssh/id_rsa"
    description: "SSH private key"
  config_value:
    from: "file ./etc/gert/config.json#/database/host"
    description: "Database host from config file"
```

The path after `file.` is the file path. An optional JSON Pointer fragment (`#/path/to/field`) extracts a nested value from a JSON or YAML file. Without a fragment, the entire file contents are returned as a string.

15.4.3 prompt

The interactive prompt provider is the default resolution strategy. It is also the fallback for any failed provider resolution (unless overridden).

```
inputs:
  server_name:
    from: prompt
    description: "Server to check"
```

In `real` and `debug` modes, the host presents a terminal prompt to the operator. In `dry-run` and `replay` modes, the prompt provider reads from the scenario's `inputs.yaml` instead. In non-interactive environments where no scenario is available, the prompt provider fails with an informative error directing the operator to supply the value via `-var` or an explicit provider.

15.4.4 workspace

Reads values from the workspace configuration file (`.gert/config.yaml`).

```
inputs:
  default_region:
    from: workspace.defaults.region
    description: "AWS region from workspace config"
```

The path after `workspace.` is a dot-notation key into the workspace configuration object. This allows teams to centralise defaults without repeating them in every runbook.

15.5 Provider Composition

A `from:` binding may declare a priority-ordered chain of providers. The host attempts each provider in order and uses the first successful result.

```
inputs:
  api_token:
    from:
      - env.ACME_API_TOKEN          # Try env var first
      - vault.secret/acme/api-token # Fall back to Vault
      - prompt                      # Final fallback: ask the operator
    description: "API token for Acme service"
```

Chaining is evaluated left-to-right. A provider is considered to have failed if it returns an error, if the binding key is absent from the resolved map, or if the resolved value is an empty string (unless the input declares `allow-empty: true`).

15.6 Provider Lifecycle

Process model. Providers that use the `stdio-jsonrpc` or `mcp` transport are *persistent*: the host spawns each provider process once at the start of the pre-flight resolution phase and keeps it alive for the entire run. This avoids repeated startup latency for providers that need to authenticate or maintain session state (e.g., Vault token renewal, PagerDuty OAuth).

Providers using the `stdio` transport are *per-resolution*: the host spawns a new process for each resolution request.

Startup. Provider processes are started lazily on first use, following the same `ready-pattern` / timeout mechanism as `stdio-jsonrpc` tools (Section 6.3).

Shutdown. At run completion (success, failure, or cancellation) the host iterates over all running provider processes and sends the method declared in `transport.startup.shutdown-method` (default: `shutdown`). Providers are given a 5-second grace period to clean up before the host sends `SIGTERM`, then `SIGKILL` after a further 5 seconds.

Crash handling. If a provider process crashes during a resolution call, the host logs a structured `provider.crashed` trace event and applies the input's `fallback:` strategy for all bindings that were in-flight at the time of the crash.

Chapter 16

Observability and Diagnostics

Operational visibility is essential for production systems. This section covers structured logging format, metric emission points, distributed tracing integration (OpenTelemetry), and the structured diagnostics model for extension and tool interactions. It makes actionable the hardening claims from § 7 by specifying concrete signals operators can use to debug failures, detect anomalies, and audit compliance.

16.1 TODO

This section is under active authoring.

References

- [1] Amazon Web Services. *AWS Step Functions Developer Guide*. State machine-based workflow orchestration with error handlers and human approval tasks. 2024. URL: <https://docs.aws.amazon.com/step-functions/> (visited on 04/18/2026).
- [2] Amazon Web Services. *AWS Systems Manager Automation Runbooks*. YAML/JSON automation documents with approval gates, cross-account execution, and rate controls. 2024. URL: <https://docs.aws.amazon.com/systems-manager/latest/userguide/automation-documents.html> (visited on 04/18/2026).
- [3] Anthropic. *Model Context Protocol (MCP) Specification*. Protocol for connecting AI models with external data sources and tools. 2024. URL: <https://modelcontextprotocol.io/> (visited on 04/18/2026).
- [4] Argo Project (CNCF). *Argo Workflows Documentation*. Kubernetes-native containerized workflow engine with YAML/JSON CRD-based definitions. 2024. URL: <https://argoproj.github.io/workflows/> (visited on 04/18/2026).
- [5] Charm. *Bubble Tea — A Go Framework for Building Terminal UIs*. The Elm Architecture for building terminal user interfaces in Go. 2024. URL: <https://github.com/charmbracelet/bubbletea> (visited on 04/18/2026).
- [6] Cloud Native Computing Foundation. *Jaeger Documentation*. Distributed tracing platform compatible with OpenTelemetry. 2024. URL: <https://www.jaegertracing.io/docs/> (visited on 04/18/2026).
- [7] Cloud Native Computing Foundation. *Kubernetes Documentation*. Container orchestration platform with admission controllers and RBAC. 2024. URL: <https://kubernetes.io/docs/> (visited on 04/18/2026).
- [8] IBM Corporation. *Delivering Consistency and Automation with Operational Runbooks*. Deep dive into IBM Runbook Automation service, blending analytics with automation for IT operations. IBM Redbooks, 2022.

- [9] *ISO/IEC 27001:2022 — Information Security Management Systems*. Tech. rep. International standard for information security management, including audit requirements (A.12.4). International Organization for Standardization, 2022.
- [10] JSON-RPC Working Group. *JSON-RPC 2.0 Specification*. Remote procedure call protocol encoded in JSON. 2010. URL: <https://www.jsonrpc.org/specification> (visited on 04/18/2026).
- [11] G. Klyne and C. Newman. *RFC 3339 — Date and Time on the Internet: Timestamps*. RFC 3339. Internet Engineering Task Force (IETF), 2002. URL: <https://www.rfc-editor.org/rfc/rfc3339> (visited on 04/18/2026).
- [12] Microsoft. *Visual Studio Code Extension API*. API for building VS Code extensions with versioned contracts. 2024. URL: <https://code.visualstudio.com/api> (visited on 04/18/2026).
- [13] *NIST Special Publication 800-53 — Security and Privacy Controls for Information Systems and Organizations*. Tech. rep. Catalog of security and privacy controls with principle of least privilege guidance. National Institute of Standards and Technology, 2020. URL: <https://csrc.nist.gov/publications/detail/sp/800-53/rev-5/final> (visited on 04/18/2026).
- [14] Open Policy Agent (CNCF). *OPA Gatekeeper Documentation*. Policy controller for Kubernetes using OPA to enforce cluster-wide policies. 2024. URL: <https://open-policy-agent.github.io/gatekeeper/> (visited on 04/18/2026).
- [15] Open Policy Agent (CNCF). *Open Policy Agent (OPA) Documentation*. General-purpose policy engine with Rego language for authorization and compliance. 2024. URL: <https://www.openpolicyagent.org/docs/> (visited on 04/18/2026).
- [16] *OpenTelemetry Specification*. Tech. rep. Specification for distributed tracing, metrics, and logs with W3C Trace Context propagation. Cloud Native Computing Foundation (CNCF), 2023. URL: <https://opentelemetry.io/docs/specs/otel/> (visited on 04/18/2026).
- [17] PagerDuty. *PagerDuty Runbook Automation*. SaaS solution integrating runbook automation with incident response (formerly Rundeck). 2024. URL: <https://www.pagerduty.com/platform/automation/> (visited on 04/18/2026).
- [18] Prefect Technologies. *Prefect Documentation*. Python-native workflow orchestration with dynamic DAGs and hybrid deployment flexibility. 2024. URL: <https://docs.prefect.io/> (visited on 04/18/2026).

- [19] “Runbook Engineering and SOP Design in High Availability Environments”. In: *International Journal of Scientific Research in Engineering and Technology (IJSRET)* 10.6 (2023). Explores principles of runbook engineering for SRE teams with focus on auditability, compliance, and MTTR reduction.
- [20] *SOC 2 — Trust Services Criteria*. Tech. rep. Auditing standard for security, availability, processing integrity, confidentiality, and privacy. American Institute of Certified Public Accountants (AICPA), 2023. URL: <https://www.aicpa.org/interestareas/frc/assuranceadvisoryservices/socforserviceorganizations.html> (visited on 04/18/2026).
- [21] Stretchr. *Testify — A Toolkit with Common Assertions and Mocks for Go*. Go testing library with assertions, mocking, and test suites. 2024. URL: <https://github.com/stretchr/testify> (visited on 04/18/2026).
- [22] Temporal Technologies. *Temporal Workflow Engine Documentation*. Durable execution with saga pattern support, compensation activities, and deterministic replay. 2024. URL: <https://docs.temporal.io/> (visited on 04/18/2026).
- [23] U.S. Department of Health and Human Services. *HIPAA Security Rule — 45 CFR Part 164, Subpart C*. Security standards for protected health information (ePHI) including audit controls. 2013. URL: <https://www.hhs.gov/hipaa/for-professionals/security/index.html> (visited on 04/18/2026).
- [24] *W3C Trace Context Specification*. Tech. rep. Standard for distributed trace context propagation via traceparent and tracestate headers. World Wide Web Consortium (W3C), 2020. URL: <https://www.w3.org/TR/trace-context/> (visited on 04/18/2026).
- [25] A. Wright et al. *JSON Schema: A Media Type for Describing JSON Documents*. Internet Draft. Draft 2020-12. JSON Schema, 2020. URL: <https://json-schema.org/specification.html> (visited on 04/18/2026).
- [26] Zipkin. *Zipkin Documentation*. Distributed tracing system for gathering timing data and troubleshooting latency. 2024. URL: <https://zipkin.io/> (visited on 04/18/2026).